DR-DOS Multitasking API Guide

Caldera OpenDOS / DR-DOS Multitasker Programming Interface

Caldera, Inc.
Archived HTML documentation converted for reading

Table of Contents

Caldera DR-DOS Multitasking API Guide

Contents

Chapter 1 - Multitasker Structure and Functions
Chapter 2 - Multitasking API
Appendix A - Error Codes

Disclaimer

Caldera, Inc. makes no representations or warranties with respect to the contents or use of this manual, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Caldera, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

Further, Caldera, Inc. makes no representations or warranties with respect to any DR-DOS software, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Caldera, Inc. reserves the right to make changes to any and all parts of DR-DOS software, at any time, without any obligation to notify any person or entity of such changes.

Trademarks

Caldera, Inc. has made every effort to supply trademark information about company names, products, and services mentioned in this manual. The following list of trademarks was derived from various sources.

Caldera and Caldera DR-DOS are registered trademarks of Caldera, Inc.

Intel is a registered trademark and 386 is a trademark of Intel Corporation.

Copyright © 1994, 1997 Caldera, Inc. All rights reserved. No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of the publisher.

Chapter 1 - Multitasker Structure and Functions

Contents of this chapter

Introduction
Core Modules
Supervisor Module
Real Time Monitor Module
Domain Manager Module
High Level Modules
Virtual Machine Module
Task Manager

Introduction

This manual describes the programming interface for the multitasking 80386 kernel. This kernel creates multiple virtual DOS machines at the request of the user who controls it through the Task Manager. The user may run a standard DOS application in each of these virtual DOS machines, but may also run applications that are aware of, and take advantage of, the multitasking. This manual tells you how to write such applications, and also multitasking-aware device drivers.

The Task Manager can also run on a single-tasking kernel as a task switcher. It has its own API that allows applications and device drivers to control and respond to task switches. If it finds that it is not running on the multitasking kernel, an additional switcher-specific API can be called by applications. The Task Manager API is described in the DR-DOS System and Programmer's Guide.

Some of the multitasking kernel's features relate directly to those of the 80386 processor. This is particularly the case with the low level functions intended only for the use of programmers of device drivers and system extensions. We use Intel's terminology and direct you to Intel* Corporation's 386 DX Microprocessor Programmer's Reference Guide for descriptions of these features, including memory paging, exception handling, Task State Segments (TSSs) and I/O bitmaps.

Multitasking applications need to call some of the kernel's functions, but many of them are intended only for the system's internal use. To simplify the life of those of you who are writing applications that do not perform low-level processor-specific and operating system-

specific operations, we have flagged each function as an application program function (named X_function) or system program function (named Z_function). If you are writing applications you can probably ignore all functions whose names start with Z_.

The major facilities that applications can use include

- Multithreading
- Synchronization functions
- Queues for inter-task communication
- Allocation of extended memory and descriptors for use in protected mode

The terminology used to describe multitasking can be confusing. There are multiple virtual DOS machines, each running an application as a task. A virtual machine and the task in it are called a domain. Each task has one or more threads of execution or processes, each of which has a state represented by its current set of processor registers. Each process must have its own stack, but may share code and static data with other processes.

Each domain has some memory that is private to it, but a domain can also access some shared or global memory. The private memory initially has the same contents as that of the parent domain, but execution of the task can change this without affecting the parent. This private memory is called instance memory and the data in it instance data since each instance of a domain has its own copy.

At any time only one process of one task is actually executing. The system keeps an image in memory of the current registers of all other processes, which it uses to load the processor's registers when it dispatches the corresponding process.

The kernel consists of a number of cooperating modules. Each module provides a complete and distinct set of services, and we therefore describe the kernel module by module. Figure 1-1 shows the internal structure of the core of the multitasking kernel, and Figure 1-5 on page 1-20 shows how the other modules relate to it.

Core Modules

The core consists of six modules.

Supervisor	Real Time Monitor
Memory Manager	Domain Manager
Module Manager	Exception Handler

Figure 1-1
Core Modules

The modules which have functions you can call are

- Supervisor (SUP), which initializes the system and controls important global data structures.
- Real Time Monitor (RTM), which manages thread-related activities including context-switching. It also has the queue, mutual-exclusion zone and flag functions. (Flags are also known as semaphores.)
- Memory Manager (MEM), which maintains the pools of available memory and descriptors.
- Domain Manager (DOM), which provides separate domains for each virtual machine to run in. The other modules within the core handle exceptions and interrupts, and maintain jump tables for inter-module calls and calls from applications.

Supervisor Module

The Supervisor module (SUP) performs a variety of jobs on behalf of the other modules. The functions that applications can call are listed in Table 2-2 on page 2-2. The most important is the multitasker installation check, which you must call before you can assume that any of the other calls are supported.

Real Time Monitor Module

The Real Time Monitor module (RTM) includes the dispatcher, and maintains all lists of processes and their states. RTM also includes the queue and mutual-exclusion zone (mutex) functions. Queues are used to communicate between processes and to synchronize activities. Mutual-exclusion zones give exclusive use of a shared resource to one process at a time. System extensions may also use the RTM flag functions.

Processes

Processes, or threads, are the active objects that execute code. Table 2-3 on page 2-3 lists the functions you call to create and manage processes. Initially, there is just one process running. When the user tells the Task Manager to create a new task, or when a process calls X_PCreate, a new process is created. The new process is said to be the child of the process that called X_PCreate.

Each process has a set of attributes.

- Name. An eight character name by which the user may identify processes.
- Handle. A number by which the process is identified to the system.
- Status. Ready to run or waiting for some event or resource.
- Priority.
- Flags. Process type, including System and Keep flags.
- Parent. The handle of the process that created this one. Processes can be ready to run, suspended, waiting on a queue, or waiting for a specified time. Just one process is actually running at a time.

The list of processes that are ready to run is ordered by their priority. When a process is created, it is given a priority. Every clock tick, the dispatcher runs the highest priority ready process for one tick, unless it is in the foreground domain when it is given more - typically five ticks. You can call X_TickGet to find out the clock tick period (typically one eighteenth second) and Z_TicksSet to set the number of ticks given to tasks in the foreground domain. If several processes have the same priority and are all ready, they share processor time in a round-robin manner. The Domain Manager and Virtual Machine manager ensure that the running process sees the correct environment.

Applications can call RTM functions to wait for some number of clock ticks or to force the dispatcher to run (just as though there had been a clock tick). They can also ask RTM to change their own priority, but this may upset the round robin scheduling, which relies on all applications having the same priority. Changing priority should be done only if it is found to be essential.

If you must perform some tasks within a specific time, or do something that makes the computer's state temporarily unstable, you can call X_CritEnter to force the dispatcher not to run while the application is in the critical region. Although interrupts are not disabled, no other process can run while your process is in a critical region, so please use this facility only if you cannot achieve the same results through interrupts and mutexes.

Processes have eight character names. These are not used by the system, but can be displayed to the user to help identify tasks. Internally, processes are identified by their 32-bit handles. These handles are global, so they may be passed to processes in any domain. For the duration of a process, its handle is unique, but a handle may be reused if a process terminates and another is created. Be careful to ensure that a handle still refers to the process you wish to specify.

The handle should not be used as a pointer, but only as a parameter to system calls. There are calls to get the name, priority, status and domain of a process, given its handle. Calls that require a process handle allow the value of 0 to imply the calling process.

When a process is created, its parent is told the child's handle, and a process can get its own handle or a list of all current process handles. You can obtain the name of any process given its handle, but you cannot obtain the handle of a process given its name. There is no guarantee or requirement that process names are unique.

Further details of the possible values of priority, flags and status are described under the system calls you use to get and/or set them; these calls are listed in Table 2-3 on page 2-3.

Queues

A queue is a fixed length buffer in global memory that can contain a certain number of messages. Each message is of a fixed length. You specify the message length and queue

capacity when you create the queue. You also specify the queue's name, which must be unique. Table 2-4 on page 2-3 lists the functions you call to create and use queues.

Any process that knows a queue's name can open it and then send messages to or read messages from it, but the most common use of queues involves just one process reading from any one queue. The content of a message is entirely a matter for the applications reading and writing it.

Although each queue's buffer will hold a fixed number of messages, processes that attempt to read an empty queue or write to a full one are suspended, so the buffer capacity and sequence of events is hidden from them. To writers, the queue appears to have an unlimited capacity. To readers it seems always to have a message. If a process does not want to be suspended, it can make the read or write conditional on immediate success. Also, you can read a message without removing it from the queue. This is called a non-destructive read.

Figure 1-2 shows the various states a queue may be in and the transitions between them. When you create a queue it is empty. Any number of processes may wait to read from an empty queue. When a process at last writes a message to the queue, the process which has been waiting longest is woken and reads the message.

Figure 1-2
Queues

A similar sequence of events takes place if processes try to write to a full queue. The writing processes are suspended. When a process reads from the queue, there will be space for one more message in the queue buffer. The message from one waiting process is immediately written into this space, and the writing process becomes ready to run again.

Queues are thus a very convenient means to synchronize processes, as well as to pass information between processes.

Mutual Exclusion Zones (Mutexes)

Mutual exclusion zones ensure that only one process at a time is performing some activity. This is one of the most common requirements when multiple processes may be running asynchronously with respect to one another.

A mutual exclusion zone is not necessarily a physical entity such as a section of code. It can be any activity that consists of two or more actions that must be completed as a unit because they require exclusive access to a shared resource. Examples of such activities and resources include shared non-reentrant code, data base updating and printing.

Mutual exclusion is achieved through the use of mutexes, a special type of queue. They have several features that enhance their usefulness.

- Automatic naming. When an application creates a mutex, it is given a unique name and a handle (a number). Other applications that wish to share the resource protected by the mutex must know the handle number.

- Queuing of processes waiting to use the resource. As with normal RTM queues, any number of processes may wait to enter a mutex zone.
- Multiple entry. The process that is in the mutex zone may reenter it. If it does so, a count of the number of times it has entered is incremented and the zone is not relinquished to another process until the owning process has exited the same number of times it entered. This allows, for instance, recursive calls to code that enters a mutex zone.
- Protection against termination. When a process terminates, the system automatically makes it exit from any mutex zones it may be in. This ensures that the zone will not be locked forever. Figure 1-3 shows the states a mutex may be in and the transitions between them. For simplicity it does not cover the case of the owning process reentering a mutex.

  Table 2-6 on page 2-5 lists the functions you call to create and use mutexes.

  Figure 1-3
  Mutex zones

  A mutex comes into existence when an application calls X_MXCreate. The mutex is automatically given a unique name and a handle that is used to identify it in all subsequent calls.

  Applications call X_MXEnter to gain exclusive use of the zone. If the zone is currently Free, its state is changed to InUse and the application continues. If another process is in the zone, the second process goes into the queue. If the process calling X_MXEnter is already in the zone, its count is incremented.

  When ready to release the resource, the owning application calls X_MXExit. If another process is waiting to enter the zone, it becomes ready to run and is dispatched immediately if its priority is high enough.

  Note that for each call to X_MXEnter you must call X_MXExit once.

  Being in a mutex guarantees that no other process can enter the same mutex, but it gives no assurance that you will get the CPU time you may need. You can call the RTM Critical Region functions X_CritEnter and X_CritExit to do this.

Flags

Flags have been provided to allow Interrupt Service Routines (ISRs) to signal asynchronous events to applications. The quickest way for an ISR to wake up a synchronous process is to call Z_FlagSet.

Table 2-6 on page 2-5 lists the functions you call to create and use flags.

Only one process can wait on any given flag.

Before you can use a flag you must ask RTM to allocate one to you. There are a limited number of flags in the system, so use them sparingly. Each flag is identified only by its number.

The process that is waiting for the event should call Z_FlagWait. If the flag is in its Off state, the process will sleep until the ISR calls Z_FlagSet or until an optional timeout expires. If the flag has already been set, the process continues immediately.

If several interrupts occur before the process calls Z_FlagWait, the flag remains in its On state with a count of the number of "missed" interrupts. The flag stays in the On state until the corresponding number of Z_FlagWait calls has been made. An error code is returned both to the ISR and to the process that calls Z_FlagWait, to show that overrun has occurred.

Figure 1-4
Flag states

Memory Manager Module

The Memory Manager module (MEM) and Domain Manager module (DOM) work together to allocate and map memory and descriptors so that each application can have the resources it needs. Many of the functions in MEM are needed only by other parts of the operating system. Table 2-7 on page 2-6 lists the functions in MEM.

Applications will not normally need to call MEM directly. If they need to allocate memory or descriptors, they should do so through higher level portable interfaces, such as XMS, VCPI and DPMI, which in turn call MEM.

Domain Manager Module

A major capability of the operating system is its ability to run multiple DOS applications simultaneously. Each application sees a real mode 8086-like environment of its own, complete with an operating system, device drivers, keyboard, mouse, screen and so on. This environment is called a virtual machine, because it does not exist physically. The combination of a virtual machine and the application running in it is called a domain. Domains are created and managed by the Domain Manager (DOM). Table 2-8 on page 2-8 lists the functions you call to create and control domains.

When the kernel starts up, only one domain is created. It is normally the Task Manager which, in response to hot-key combinations pressed by the user, instructs DOM to create domains, delete them and switch selected domains to the foreground (that is, attach them to the keyboard, mouse and screen).

DOM actually performs only the lower level functions needed to create virtual 8086 and virtual DOS machines. Higher level functions such as virtualizing a VGA are carried out by the VM module.

A virtual machine is achieved by mapping the required physical memory into the first four megabytes of linear address space, setting traps to intercept I/O, then putting the processor into virtual 8086 mode. Virtual 8086 mode is practically indistinguishable from real mode as far as DOS applications are concerned. The required physical memory is a combination of unshared instance memory and shared global memory. The virtualization includes very sophisticated exception handling that can distinguish between the many

sources of exceptions and interrupts. By trapping, for example, attempts to read from the screen memory, it allows applications to continue to run in the background. Each application "sees" a full screen. Only the foreground application's screen is visible to the user, but an application can force urgent messages to be displayed immediately, even if its domain is in the background, by calling the Virtual Machine module (VM); see page 1-21 for a description of VM.

I/O Bitmaps

Each domain has its own I/O bitmap. The bitmap has a bit for each I/O port address. The bits corresponding to virtualized ports are set. This causes a General Exception whenever an application tries to read from or write to the port. The system recognizes that the exception was caused by an I/O operation, so it invokes the Virtual Machine module's I/O exception handlers, which make sure the port ownership rules are obeyed; see "Virtual Machine Module" on page 1-21 for a description of the rules.

If you are writing system software that similarly needs to trap accesses to other I/O ports, you can call DOM functions to read, set and reset bits in the I/O bitmap of any domain. These functions' names all start Z_IOBitmap...

See the description of Z_HandlerGenEx for a full explanation of exception and interrupt handling, including the use of installable handlers.

Instancing

Before the Task Manager is loaded, only one domain exists. In its initialization, Task Manager must determine which memory areas should be instanced. It uses several methods to do this, including making an INT 2FH with AX=4B05H to ask global programs to identify their instance data. Programs that need to instance their data in a multitasking environment must hook INT 2FH and process it appropriately. Any memory owned by device drivers and other resident programs that they do not identify as instanced is assumed to be global. Programs loaded after the Task Manager is running will be loaded into local memory, that is, instance memory within one domain only.

The Task Manager informs DOM which addresses are to be instanced when it has found them all.

Memory instancing uses 80386 page tables for large areas and software-controlled swapping for small ones. Each domain has its own page table for the first 4 MB of memory. These page tables point to globally shared pages and unshared instanced pages. A context switch that includes a domain switch involves four steps to map memory.

1. Saving the instanced portions of global pages into a data area owned by the outgoing domain
2. Switching to the new domain's base page table
3. Copying the global portions of instance pages from the previous domain's copy
4. Loading the instanced portions of global pages from a data area owned by the incoming domain

As well as mapping memory, a domain switch loads the I/O bitmap and exception handlers for the incoming domain, because these may vary from domain to domain.

Domain Screen Switching

Applications do not normally need to be aware of which domain owns the physical screen, and which domains are running in the background. Unless you are writing system software, you can skip this section.

Global programs that need to be notified of domain screen switches should hook INT 2FH and be alert for the function Build Notification Chain (AX=4B01H), which will be the first indication they have that the Task Manager is active. This informs the kernel of the program's callback function's address, and informs the program of the kernel's service functions' address.

While the Task Manager is active, programs can call Detect Switcher (INT 2FH / AX=4B02H) and then Hook Notification Chain to achieve the same thing.

Programs can call the service functions to perform the following:

- Get the version number of the Task Manager.
- Test if a memory region is local or global or a mixture.
- Hook or unhook themselves from the notification chain (not very necessary, as the chain is rebuilt from scratch every time a domain goes into the foreground).
- Query API support. This allows for different providers of the same asynchronous API to determine which of them should take control. Programs on the notification chain are notified by the kernel when a different domain is switched into the foreground and when a domain is created or destroyed.

High Level Modules

On top of the core modules, the multitasker has several higher level modules, shown in Figure 1-5 and described in the rest of this chapter.

Figure 1-5
High Level Modules

Of the modules shown in Figure 1-5 only the Virtual Machine (VM) has an API that is unique to the multitasker, though the Task Manager's API is reduced when running on the multitasker. Other modules have no API, or behave the same whether running on the multitasker or raw single tasking DOS, with or without the Task Manager. Their descriptions are therefore not included in this manual.

Virtual Machine Module

The Virtual Machine module (VM) completes the virtualization of a PC from the application's perspective. Its major tasks are

- Ensuring that an application does not use hardware which is being used by another task
- Saving/restoring the virtual machine when access to the hardware is to be switched between domains
- Displaying pop-up messages
- Eliminating virtualization overheads when the Task Manager is not loaded (that is, when there is just one task running) Data associated with each virtual machine includes
- Hardware interrupt vectors
- ROM BIOS area
- Memory areas dual-ported with hardware, such as video memory
- I/O ports The Task Manager calls VM when the user creates, destroys or switches between tasks. Applications can call VM to display messages when they are in the background. TSRs may display messages on their domain's virtual console. VM also has functions that applications can call to set up the serial and parallel ports.lists the functions you can call in VM.

Pop Up Messages

A TSR running in a domain can call VM to ask it to prepare for, and optionally display, a pop-up message on the TSR domain's virtual console. Similarly, an application whose domain is currently in the background may display a message over the foreground screen. In both cases VM saves the previous screen contents and restores them after the message has been shown.

The screen may have been in a graphic mode, so VM clears it and puts it into the default text mode, allowing you to display text by INT 10 or direct screen writes.

Domain Message Mode

Use Domain Messages if your program is a TSR or thread that does not own its domain's screen, and therefore needs to save the screen mode and contents before displaying a message. VM has two related calls: X_MsgDomEnter and X_MsgDomDisplay. They both save the call parameters, put the domain into Domain Message Mode and return immediately to the caller. The message has not been displayed, and nor is the screen in text mode at this point, unless the domain is in the foreground.

If the domain is in the background, its application can continue to write to the virtual screen as normal. When the domain comes to the foreground, VM saves the screen mode and contents, then either displays a message string you passed it (if you called X_MsgDomDisplay) or calls a function you specified (if you called X_MsgDomEnter).

The domain is in the foreground when your message display function is called, and can therefore use the keyboard to take the user's instructions. When the user has responded and you want to remove the message, call X_MsgDomExit. VM restores the original screen mode and contents, and turns off Domain Message Mode.

While in Domain Message Mode, the user can switch a different domain to the foreground. If so, your message will not be seen again until its domain is again in the foreground. X_MsgDomEnter calls your message display function each time the domain comes to the foreground.

Global Message Mode

Use Global Message Mode if your application needs urgently to display a message when its domain is, or may be, in the background. Again, VM has two related calls: X_MsgGlobalEnter and X_MsgGlobalDisplay. They immediately bring the domain to the foreground, save the screen mode and contents, and put the screen into its default text mode. At this point, X_MsgGlobalEnter returns to the caller, whereas X_MsgDomDisplay first displays a message string you passed it.

Because the domain is now in the foreground, you can read the keyboard and output to the screen (though with some restrictions described on page 2-121). When the user has responded and you want to remove the message and return to the background, call X_MsgGlobalExit.

While in Global Message Mode the Task Manager is disabled, so the user cannot avoid seeing the message.

Serial and Parallel Ports

The standard ports known to the ROM BIOS are initially set up to use their default port numbers and IRQs. You can call VM to change these and to tell it the port numbers and IRQs of COM and LPT ports (up to COM4 and LPT3), which the BIOS does not know about. These values are physical and therefore global.

While the Task Manager is running, VM virtualizes the serial and parallel ports. The method it uses to decide which domain can use the physical ports is based on timeouts and traps, as follows.

Timeout values (t) are initially set to 5 seconds for a COM port, and 15 seconds for a printer port, though you may call VM to change the timeout on any port. The operation of timeouts is:

1. Initially, VM traps all accesses of all virtualized ports (those that have a non-zero base address and timeout).
2. The first domain to access a given COM or PRN port becomes the "owner" of that port, thus other domains cannot use the port. VM stops trapping the port for a time of t/2.
3. When that timeout expires, VM sets its trap again, and restarts another timeout of t/2. If the owner accesses the port within that time, VM immediately clears its trap and goes back to Step 2, otherwise Step 4 is performed.
4. The port is freed, and will be given to the next domain to access it.

The specified timeout is therefore the maximum time which can elapse between the last access by the owner and the point at which the port is freed. The minimum time is t/2. The

timeout value should be set to at least twice the maximum interval expected between port accesses while the port is being used.

A timeout value of 0 disables virtualization of the port. Any domain may access it freely at any time.

A timeout value of -1 sets an "infinite" timeout; thus, once the port is given to a domain, it never becomes free again, unless the domain is deleted.

Task Manager

The Task Manager controls switching between multiple DOS applications by giving the VM and DOM modules the appropriate sequences of commands. It accepts instructions from the user via keyboard hot-key combinations. It can also be controlled by software, using its native API (INT 2FH / AH=27H), though only the functions listed in Table 2-10 on page 2-11 are available when the Task Manager is running on the multitasking kernel.

The Task Manager also supports the industry-standard INT 2FH / AH=4BH API for task switchers with its corresponding service and notification functions. This interface is provided to support global programs written to run on other DOS task switchers. Although these functions are not described in this manual, they are listed in Table 2-10 on page 2-11 to Table 2-13 on page 2-13.

When the Task Manager is not present, the kernel does not respond if an application or device driver tries to detect the presence of a switcher (INT 2FH / AX=4B02H) or asks for a switcher ID for itself (INT 2FH / AX=4B03H). When the Task Manager is present, the kernel does respond to these requests. However, the Task Manager does not itself call INT 2FH / AX=4B02H, so it cannot be run if a task switcher is already present.

---

info@caldera.com
Copyright © 1994, 1997, Caldera, Inc. All rights reserved.

Chapter 2 - Multitasker API

Contents of this chapter

General Information
Installation Check
Functions

General Information

Real mode applications call the multitasker's native API via the general-purpose INT 2F Multiplex Interrupt, with the multitasker being specified by the value 2780H (hex) in register AX.

NOTE: For brevity, the value AX=2780H is not listed in the entry parameters for each function.

The value in register CX identifies the individual functions, with CL being the module number (see Table 2-1) and CH the function within a module. The first call you make must be the Installation Check (see "Installation Check" on page 2-14), to verify that the multitasker is present.

Many of the INT 2F / AX=2780H calls take parameters in other registers. Some of these use the 32-bit general purpose registers. This should cause you no difficulty so long as you direct the assembler to accept 32-bit registers and operands. Parameters that are pointers are sometimes passed as 32-bit linear addresses, not segment:offset pairs. You can convert a segment:offset pair to a linear address by calculating segment*16+offset.

All functions return with the carry flag (CF) clear and CX=0 if successful, otherwise with CF set and the error code in CX. Values returned in EBX are copied into EAX, so you may use either register. All functions preserve the values in all other registers and flags. Where DS:EDX is used to point to a parameter block, we use [EDX] to mean "the memory pointed to by EDX."

The function names are equated to their values in the supplied assembly language include file FUNCTION.DEF, summarized in the following tables.

Error codes are equated in ERR.DEF and listed in Table A-1 on page A-1. INT 2FH with AX=2780H and an illegal module number or function number in CX will return an error code of E_BadModule or E_NotImplemented respectively.

The following tables list all the functions within each module, ordered by module number and function group. Within each table, functions are ordered alphabetically.

Table 2-1
Multitasker Module Numbers

CL Hex	Module
01	Supervisor (SUP)
02	Real Time Monitor (RTM)
03	Memory (MEM)
04	Domains (DOM)
05	Virtual Machines (VM)

Table 2-2
SUP Functions

CX Hex	Function Name	What it does	Page
1101	Installation check	Check if multitasker installed	2-14
0201	Z_ModuleReg	Register OS Module	2-112
0501	Z_MoveReal	Move from real memory to extended memory	2-114
0601	Z_Reboot	Reboot	2-198

Table 2-3
RTM Process Functions

CX Hex	Function	What it does	Page
2502	X_CritEnter	Enter Critical Region	2-15
2602	X_CritExit	Exit Critical Region	2-17
0102	Z_PCountGet	Get count of dispatches	2-125
0E02	X_PCreate	Create a new process	2-126
0B02	X_PDelay	Go to sleep for some clock ticks	2-130
0C02	X_PDispatch	Force a dispatch	2-131
1002	X_PHandleGet	Get my process handle	2-132
2802	Z_PHandleListGet	Get list of process handles	2-133
2902	Z_PNameGet	Get a process' name	2-135
2C02	Z_PPriorGet	Get this process' priority	2-136
0F02	Z_PPriorSet	Set this process' priority	2-132
2A02	Z_PStatusGet	Get a process' status	2-138
2302	X_PSuspend	Suspend a process	2-140
1102	X_PTerm	Terminate a process	2-141
1702	X_PTermOff	Disable terminating	2-143

1802	X_PTermOK	Reenable terminating	2-144
2402	X_PUnsuspend	Release a process	2-145
3D02	X_TickGet	Get clock tick period	2-213
3C02	Z_TicksSet	Set foreground slices	2-214

Table 2-4
RTM Queue Functions

CX Hex	Function	What it does	Page
3F02	X_QClose	Close a queue	2-172
0402	X_QCreate	Create a new queue	2-173
0602	X_QDelete	Delete a queue	2-176
3402	Z_QFlagsGet	Get a queue's flags	2-179
4502	Z_QFlagsSet	Set a queue's flags	2-179
2D02	Z_QHandleListGet	Get a list of queue handles	2-180
2F02	X_QMsgLenGet	Get a queue's message length	2-182
3002	X_QMsgMaxGet	Get a queue's message capacity	2-183
3302	X_QMsgNumGet	Get the number of messages in a queue	2-184
2E02	Z_QNameGet	Get a queue's name	2-133
0502	X_QOpen	Open a queue for read/write	2-188
0702	X_QRead	Read a message from a queue	2-189
0802	X_QReadC	Read a message if there is one	2-191
1F02	X_QReadNDC	Peek at a message if there is one	2-192
3202	Z_QReaderGet	Get handle of process waiting to read	2-193
0902	X_QWrite	Write a message to a queue	2-194
0A02	X_QWriteC	Write a message if there is space	2-196
3102	Z_QWriterGet	Get handle of process waiting to write	2-197

Table 2-5
RTM Mutex Functions

CX Hex	Function	What it does	Page
3702	X_MXCreate	Create a new mutex	2-92
3802	X_MXDelete	Delete a mutex	2-94
3902	X_MXEnter	Enter a mutex zone	2-95
3A02	X_MXEnterC	Enter a mutex zone if it is free	2-97

3B02	X_MXExit	Exit from a mutex zone	2-98

Table 2-6
RTM Flag Functions

CX Hex	Function	What it does	Page
1C02	Z_FlagAlloc	Allocate a flag	2-40
1D02	Z_FlagFree	Free a flag	2-41
0302	Z_FlagSet	Set a flag	2-42
1902	Z_FlagStatusGet	Return status of a flag	2-43
0202	Z_FlagWait	Wait on a flag	2-44
2002	Z_FlagWWTO	Wait with a timeout	2-45
1E02	X_FlagsMaxGet	Return number of flags supported	2-46

Table 2-7
MEM Functions

CX Hex	Function	What it does	Page
1003	Z_DescAlloc	Allocate a descriptor	2-18
1103	Z_DescFree	Release a descriptor	2-21
1203	Z_DescGet	Get a descriptor's details	2-22
1303	Z_DescSet	Set a descriptor	2-23
2F03	Z_GateAlloc	Allocate a gate	2-50
3003	Z_GateFree	Release a gate	2-52
0C03	Z_MemAlloc	Allocate a memory block	2-99
1403	Z_MemDescAlloc	Allocate a memory block and its descriptor	2-101
1503	Z_MemDescFree	Release a memory block and its descriptor	2-103
1703	Z_MemDescResize	Resize a block	2-104
1603	Z_MemDescSizeGet	Get size of a block	2-105
0D03	Z_MemFree	Release a memory block	2-106
3103	X_MemFreeGet	Get total free memory	2-107
0F03	Z_MemResize	Resize a block	2-108
0E03	Z_MemSizeGet	Get size of a block	2-109
3203	X_MemTopGet	Get top memory address	2-110
3303	X_MemTotalGet	Get total memory size	2-111
0303	Z_PageAlloc	Allocate a given page	2-146

2903	Z_PageDomLock	Lock a page in a specified domain, with existing contents	2-147
2B03	Z_PageDomLockAny	Lock a page in a specified domain, with undefined contents	2-148
2D03	Z_PageDomLockNone	Lock a page in a specified domain, with no physical memory	2-149
2A03	Z_PageDomUnlock	Unlock a page in a specified domain	2-150
2E03	Z_PageDomUnlockNone	Unlock a page in a specified domain, physical memory not reused	2-151
2C03	Z_PageDomUnlockReuse	Unlock a page in a specified domain, physical memory reused	2-152
0B03	Z_PageFree	Free a page of memory	2-153
1803	Z_PageLock	Lock a page, with existing contents	2-154
1A03	Z_PageLockAny	Lock a page, with undefined contents	2-155
1C03	Z_PageLockNone	Lock a page, with no physical memory	2-156
1903	Z_PageUnlock	Unlock a page	2-157
1D03	Z_PageUnlockNone	Unlock a page, physical memory not reused	2-158
1B03	Z_PageUnlockReuse	Unlock a page, physical memory reused	2-159
0A03	Z_PagesAlloc	Allocate memory pages	2-160
0803	Z_PtblGet	Read page table	2-170
0903	Z_PtblSet	Write page table	2-171

Table 2-8
DOM Functions

CX Hex	Function	What it does	Page
1504	Z_DomFork	Copy this domain	2-24
2B02	Z_DomHandleGet	Get a process' domain	2-25
0204	X_DomHandleGetMy	Get current domain handle	2-26
2604	X_DomMemFreeGet	Get a domain's free memory	2-27
2804	Z_DomMemMaxGet	Get per-domain memory limit	2-28
2904	Z_DomMemMaxSet	Set per-domain memory limit	2-29
0D04	Z_DomMemRead	Read memory in another domain	2-30
2704	X_DomMemUsedGet	Get memory used by this domain	2-32
0E04	Z_DomMemWrite	Write to memory in another domain	2-33
1204	X_DomNProcessesGet	Get number of processes in domain	2-34
2104	Z_DomRootProcessGet	Get handle of domain's root process	2-35
1304	Z_DomSuspend	Suspend a domain	2-36
1604	Z_DomTerm	Terminate all processes in a domain	2-37

1404	Z_DomUnsuspend	Resume process execution in a domain	2-38
1D04	Z_EndOfInterrupt	Signal EOI	2-39
0801	X_ForeCheck	Check if domain is in foreground	2-47
0B05	X_ForeGet	Get foreground domain	2-49
0704	Z_HandlerGenEx	Install general exception handler	2-53
0804	Z_HandlerHWInt	Install HW int handler	2-59
1804	Z_HandlerHWIntDflt	Install default HW int handler	2-61
0604	Z_HandlerIOEx	Install I/O exception handler	2-62
1F04	Z_HandlerPCreate	Install process create handler	2-65
2004	Z_HandlerPTerm	Install process terminate handler	2-68
0504	Z_HandlerPageFault	Install page fault handler	2-70
0404	Z_HandlerSWInt	Install SW int handler	2-72
1B04	Z_HandlerSwapIn	Install swap in handler	2-74
1C04	Z_HandlerSwapOut	Install swap out handler	2-76
1704	Z_HandlerUnlink	Uninstall a handler	2-78
1904	Z_HandlerVHWInt	Install virtual HW int handler	2-79
1A04	Z_HandlerVHWIntDflt	Install default virtual HW int handler	2-81
0B04	Z_IOBitmapDomGet	Get a domain's I/O Bitmap entry	2-83
0C04	Z_IOBitmapDomSet	Set a domain's I/O Bitmap entry	2-85
0904	Z_IOBitmapGet	Get the current domain's I/O Bitmap entry	2-87
0A04	Z_IOBitmapSet	Set the current domain's I/O Bitmap entry	2-88
1104	Z_InstanceSet	Register instanced addresses	2-89
0F04	Z_PtblDomGet	Read a domain's Page Table	2-167
1004	Z_PtblDomSet	Write a domain's Page Table	2-169

Table 2-9
VM Functions

CX Hex	Function	What it does	Page
1505	X_MsgDomEnter	Enter Domain Message Mode	2-115
1605	X_MsgDomDisplay	Display message	2-117
1705	X_MsgDomExit	Exit mode	2-119
1805	Z_MsgFatalDisplay	Display message	2-120
1205	X_MsgGlobalEnter	Enter Global Message Mode	2-123
1305	X_MsgGlobalDisplay	Display message	2-121

1405	X_MsgGlobalExit	Exit mode	2-124
1A05	Z_ParallelBaseSet	Set port address	2-161
1C05	Z_ParallelIRQSet	Set port IRQ	2-162
2305	Z_ParallelOwnerGet	Get port owner	2-163
2405	Z_ParallelTimeoutGet	Get port timeout	2-164
0F05	Z_ParallelTimeoutSet	Set port timeout	2-165
1905	Z_SerialBaseSet	Set port address	2-199
1B05	Z_SerialIRQSet	Set port IRQ	2-200
2105	Z_SerialOwnerGet	Get port owner	2-201
2205	Z_SerialTimeoutGet	Get port timeout	2-202
0E05	Z_SerialTimeoutSet	Set port timeout	2-203
0A05	Z_TMHotKeyDisable	Stop scanning	2-205
0905	Z_TMHotKeyEnable	Restart scanning	2-206
0805	Z_TMHotKeyGet	Get task manager key	2-207
0705	Z_TMInit	Initialize task manager	2-209
0105	Z_TMLoad	Load task manager	2-211
0205	Z_TMUnload	Unload task manager	2-212
0305	Z_VMBackSet	Send VM to back	2-215
0405	Z_VMForeSet	Send VM to front	2-216
0605	Z_VMSaveDisable	Disable saving VM	2-217
0505	Z_VMSaveEnable	Enable saving VM	2-218

Table 2-10
Other INT 2FH Functions

AX Hex	Function	What it does	Page
1680	Release Time Slice	Same as X_PDispatch	2-131
1681	Enter Critical Region	Enter Critical Section Mutex	2-15
1682	Exit Critical Region	Exit Critical Section Mutex	2-17
1683		Get VM ID	
1684		Get VxD API entry point	
1685		Call a function in a different VM	
2700	Task Manager Install Check	Task manager	1-26
2701	Task Manager Status	Task manager	1-26
2702	Get EMS limit per task	Task manager	1-26

2703	Set EMS limit per task	Task manager	1-26
2705	Enable/disable direct switching	Task manager	1-26
2706	Switch Tasks	Task manager	1-26
2707	Create Task	Task manager	1-26
2708	Delete Task	Task manager	1-26
270A	Get Task ID	Task manager	1-26
270B	Get Task Index	Task manager	1-26
4B02	Switcher Install Check	Return entry point of Switcher Service Function	
4B03		Allocate a switcher ID	
4B04		Free a switcher ID	

Table 2-11
Switcher Service Functions (called through Switcher Service Functions entry point)

AX Hex	Function	What it does	
0000	Get version	Return version information	
0001	Test memory	Return local/global information	
0002	Suspend	Refuse to suspend	
0003	Resume	Nothing	
0004	Hook Notification Chain	Ask switcher to inform it of session switches	
0005	Unhook	Reverse of above	
0006	Query API Support	Find who else is out there	

Table 2-12
INT 2FH Functions called by the Task Manager

AX Hex	Function	What it does	
4B01	Build Notification Chain	Ask global programs to identify themselves. Tells them the entry point of the Switcher Service Function	
4B05	Identify Instance Data	Ask global programs to identify their instance data	

Table 2-13
Notification Functions called by the Task Manager

EAX Hex	Function	What it does	
0000	Initialize switcher	Warn of, and ask if OK to, start task switcher	

0003	Activate session	Warn of session switch (interrupts off)	
0004	Session active	Warn of session switch (interrupts on)	
0005	Create session	Ask if OK to create a session	
0006	Destroy session	Warn that a session is being deleted	
0007	Terminate switcher	Warn that switcher terminating	

Installation Check

Function

Check whether the multitasker is resident.

Entry Parameters

Register	Value
AX	**2780H**
CX	**1101H**

Returned Values

Register	Value
CX	**00H - Multitasker resident** **1101H - Multitasker not resident**
EBX	**Version (if CX=0)**

Explanation

The Installation Check call must be the first one you make, because other Multitasker functions can only be accessed if the multitasker is resident.

If the multitasker is present, its version number is returned in BX. The initial release is version 1.00, corresponding to the value 0100H in BX.

X_CritEnter (2502H)

Function

Enter a critical region.

Entry Parameters

Register	Value
CX	**2502H**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **3EH - E_CritOverrun**	 **>255 CritEnters**

Explanation

X_CritEnter disables the dispatcher, so no other process can run until you make a corresponding X_CritExit call. Interrupt service routines can run, however.

You can call X_CritEnter again while inside a critical region. If you do, you must call X_CritExit the same number of times you called X_CritEnter. The system maintains a count of the number of times you have entered, and only allows other processes to run when it reaches zero again.

The only error possible is that the count may overflow and generate the E_CritOverrun error.

Critical regions are similar to mutexes, but should be used only for truly time-critical operations, because they completely disable all other processes. While your process is in a critical region, it can call other system functions, but it cannot suspend or terminate itself. If it calls functions to do either of these they will not take effect until after the X_CritExit call.

See Also: X_CritExit, X_MXCreate

NOTE: The industry-standard call INT 2FH / AX=1681H is also supported by the multitasker, and is the preferred method.

X_CritExit (2602H)

Function

Exit from a critical region.

Entry Parameters

Register	Value
CX	**2602H**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **3DH - E_CritUnderrun**	 **No CritEnter**

Explanation

X_CritExit re-enables the dispatcher if called the same number of times as X_CritEnter. Call X_CritExit after completing the operation protected by X_CritEnter. If the clock has ticked since you called X_CritEnter, the dispatcher will be called immediately.

If you were not in a critical region when you called X_CritExit, it returns an E_CritUnderrun error.

See Also: X_CritEnter

NOTE: The industry-standard call INT 2FH / AX=1682H is also supported by the multitasker, and is the preferred method.

Z_DescAlloc (1003H)

Function

Allocate a descriptor.

Entry Parameters

Register	Value
CX	**1003H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Explanation"**

Returned Values

Register	Value	Meaning
EBX	**=DESC_PB_SEL**	**Selector**
ECX	**Error code:** **00H - No error** **31H - E_BadDesc** **32H - E_NoDesc**	 **Invalid DESC_PB_SINFO** **No free descriptors**
DS:[EDX]	**DESC_PB_SEL**	**Selector**

Explanation

Z_DescAlloc allocates a protected mode memory descriptor and returns its selector. Applications and device drivers should normally allocate their descriptors through a higher level interface: XMS or DPMS.

The parameter block is defined in STRUCS.DEF as follows:.

DESC_PBLK	struc	
DESC_PB_BASE	dd	?
DESC_PB_LIMIT	dd	?
DESC_PB_SEL	dw	?
DESC_PB_MINFO	db	?
DESC_PB_SINFO	db	?
DESC_PBLK	ends	

Before you call Z_DescAlloc, you must initialize all fields in DESC_PBLK except DESC_PB_MINFO and DESC_PB_SEL, which will be filled with the selector of the allocated descriptor when the function returns successfully. If the size exceeds 1 MB, the granularity will be set to 4 KB, otherwise the segment will have byte granularity. In either case, the DESC_PB_LIMIT you supply must be the limit (size-1) in bytes.

DESC_PB_MINFO is a packed flag byte. It is not used by Z_DescAlloc, but is used by related calls which share the same DESC_PBLK parameter block. The bits have the following meanings. Bits 6 and 7 should be reset.

Bit	Reset	Set
0	Not 4 KB aligned	4 KB aligned
1	Not 64 KB aligned	64 KB aligned
2	Pageable to disk	Locked
3	Domain local	Global
4	User access	System access only
5	Memory present	Empty address space required

DESC_PB_SINFO is also a packed flag byte. The bits have the following meanings. Other bits must be reset.

Bit	Reset	Set
0	16-bit code or data	32-bit code or data
1	Data segment	Code segment
4	Allocate from LDT	Allocate from LDT
5,6	Privilege level	Privilege level

Note that data segments are assumed to be writable and expand-up, and code segments to be non-conforming and readable. Call Z_DescFree to deallocate the descriptor when it is no longer needed.

See Also: Z_DescFree, Z_DescGet, Z_MemDescAlloc, Z_DescSet

Z_DescFree (1103H)

Function

Release a descriptor.

Entry Parameters

Register	Value
CX	**1103H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **31H - E_BadDesc**	 **Invalid selector**

Explanation

Z_DescFree releases a protected mode memory descriptor previously allocated by Z_DescAlloc or Z_MemDescAlloc. The parameter block is as described under Z_DescAlloc. Only the DESC_PB_SEL field need be filled in when you call Z_DescFree.

Note that freeing a descriptor renders its selector invalid. The selector must not be loaded into a segment register while the descriptor is freed, or reloaded at a later date.

See Also: Z_DescAlloc, Z_DescGet, Z_MemDescAlloc, Z_DescSet

Z_DescGet (1203H)

Function

Get a descriptor's base, limit and flags.

Entry Parameters

Register	Value
CX	**1203H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **31H - E_BadDesc**	 **Invalid selector**

Explanation

Z_DescGet returns a descriptor's base, limit and flags in the DESC_PBLK supplied by the caller. The parameter block is as described under Z_DescAlloc. Only the DESC_PB_SEL field need be filled in when you call Z_DescGet. The other fields will be filled in on return.

See Also: Z_DescAlloc, Z_MemDescAlloc, Z_DescSet

Z_DescSet (1303H)

Function

Set a descriptor's base, limit and flags.

Entry Parameters

Register	Value
CX	**1303H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
EBX	**Selector**	
ECX	**Error code:** **00H - No error** **31H - E_BadDesc**	 **Invalid selector or DESC_PB_SINFO**

Explanation

Z_DescSet changes a descriptor's base, limit and flags to the values in the DESC_PBLK supplied by the caller. The parameter block is as described under Z_DescAlloc. You must fill in all its fields when you call Z_DescSet. The selector is unaffected by this call.

See Also: Z_DescAlloc, Z_DescGet, Z_MemDescAlloc

Z_DomFork (1504H)

Function

Fork the current domain.

Entry Parameters

Register	Value
CX	**1504H**
DS:EDX	**Pointer to a PCREATE_PBLK parameter block; see "X_PCreate (0E02H)" on page 2-126**

Returned Values

Register	Value	Meaning
EBX	**Handle**	**Child domain's handle**
CX	**Error code:** **00H - No error** **xxH - Error codes**	**See Table A-1 on page A-1**

Explanation

Z_DomFork creates an exact copy of the current domain, then creates a process in it and starts the process executing. The process is specified in the PCREATE_PBLK which is identical to that used in the X_PCreate call described on page 2-126.

Z_DomFork may fail at any stage, which requires a limited resource to be allocated. See Table A-1 on page A-1 for a full list of error codes.

See Also: Z_DomTerm, X_PCreate

Z_DomHandleGet (2B02H)

Function

Get the domain of a specified process.

Entry Parameters

Register	Value
CX	**2B02H**
EDX	**Process handle**

Returned Values

Register	Value	Meaning
EBX	**Domain handle**	
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle**	 **Process handle is invalid**

Explanation

Z_DomHandleGet returns the handle of the domain that a specified process is in, given the process handle. The domain handle can be used in subsequent calls to the Domain Manager.

See Also: X_PCreate, X_PHandleGet, Z_PHandleListGet, Z_PNameGet, X_DomHandleGetMy

X_DomHandleGetMy (0204H)

Function

Get the current domain's handle.

Entry Parameters

Register	Value
CX	**0204H**

Returned Values

Register	Value	Meaning
EBX	**Handle**	**Handle of calling process' domain**
ECX	**Error code:** **00H - No error**	

Explanation

X_DomHandleGetMy returns the handle of the calling process' domain. This can then be used to call other functions in the Domain Module, although you can always pass a null handle (0) to imply "the current domain."

You can get the domain handle of another process, if you know its process handle, by calling Z_DomHandleGet.

See Also: Z_DomHandleGet

X_DomMemFreeGet (2604H)

Function

Get free memory size for a specified domain.

Entry Parameters

Register	Value
CX	**2604H**
EDX	**Domain handle (0=current domain)**

Returned Values

Register	Value	Meaning
EBX	**Free memory**	
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	 **Invalid domain handle**

Explanation

X_DomMemFreeGet returns the number of bytes the specified domain could allocate. This is equal to the Domain Memory Limit (see Z_DomMemMaxSet) minus the amount already allocated to the domain, or the total of free memory if that is lower.

See Also: Z_DomMemMaxGet, Z_DomMemMaxSet, X_DomMemUsedGet

Z_DomMemMaxGet (2804H)

Function

Get the domain memory limit.

Entry Parameters

Register	Value
CX	**2804H**

Returned Values

Register	Value	
EBX	**Domain Memory Limit**	
ECX	**Error code:** **00H - No error**	

Explanation

Z_DomMemMaxGet returns the Domain Memory Limit. This is the global limit on the memory that any domain may use. Initially, this limit is set to all memory, but the task manager may reduce it, typically to allow badly-behaved applications which allocate all available memory to coexist.

See Also: X_DomMemFreeGet, Z_DomMemMaxSet, X_DomMemUsedGet

Z_DomMemMaxSet (2904H)

Function

Set the domain memory limit.

Entry Parameters

Register	Value
CX	**2904H**
EDX	**New value for the Domain Memory Limit**

Returned Values

Register	Value	
EBX	**Old limit**	
ECX	**Error code:** **00H - No error**	

Explanation

Z_DomMemMaxSet sets a new value for the Domain Memory Limit. This is used to limit allocations, including VCPI allocations, so that no one application can allocate all the physical memory.

The previous value of the limit is returned in EBX.

See Also: X_DomMemFreeGet, Z_DomMemMaxGet, X_DomMemUsedGet

Z_DomMemRead (0D04H)

Function

Read memory in another domain.

Entry Parameters

Register	Value
CX	**0D04H**
DS:EDX	**Pointer to a BANKMEM_PBLK parameter block; see "Explanation"**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	 **Invalid domain handle**

Explanation

Z_DomMemRead allows programs running in one domain to read the memory of another domain. The related function Z_DomMemWrite allows them to write the memory of another domain.

You must supply all fields in the parameter block, which is defined as follows:

BANKMEM_PBLK	struc	
BANKM_PB_DOMAIN	dd	?
BANKM_PB_LENGTH	dd	?
BANKM_PB_SOURCE	dd	?
BANKM_PB_DEST	dd	?
BANKMEM_PBLK	ends	

- BANKM_PB_DOMAIN is the source domain handle.
- BANKM_PB_LENGTH is the number of bytes you want to read.
- BANKM_PB_SOURCE is the start address within the source domain.
- BANKM_PB_DEST is the start address, within your domain, of a buffer large enough to hold BANKM_PB_LENGTH bytes. Note that BANKM_PB_SOURCE and BANKM_PB_DEST are linear addresses, so may be calculated from virtual-8086 mode addresses by 32-bit arithmetic using Segment*16 + Offset.

  See Also: Z_DomMemWrite

X_DomMemUsedGet (2704H)

Function

Get size of memory allocated to a specified domain.

Entry Parameters

Register	Value
CX	**2704H**
EDX	**Domain handle (0=current domain)**

Returned Values

Register	Value	Meaning
EBX	**Allocated memory**	
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	 **Invalid domain handle**

Explanation

X_DomMemUsedGet returns the number of bytes allocated to the specified domain, including VCPI allocations.

See Also: X_DomMemFreeGet, Z_DomMemMaxGet, Z_DomMemMaxSet

Z_DomMemWrite (0E04H)

Function

Write to memory in another domain.

Entry Parameters

Register	Value
CX	**0E04H**
DS:EDX	**Pointer to a BANKMEM_PBLK parameter block; see "Z_DomMemRead (0D04H)" on page 2-30**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	 **Invalid domain handle**

Explanation

Z_DomMemWrite allows programs running in one domain to write to the memory of another domain. The related function Z_DomMemRead allows them to read from the memory of another domain.

You must supply all fields in the BANKMEM_PBLK parameter block which is defined under Z_DomMemRead.

See Also: Z_DomMemRead

X_DomNProcessesGet (1204H)

Function

Get the number of processes in a specified domain.

Entry Parameters

Register	Value
CX	**1204H**
EDX	**Domain handle (0=current domain)**

Returned Values

Register	Value	Meaning
EBX	**Number**	
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	 **Invalid handle**

Explanation

X_DomNProcessesGet returns the current number of processes in the specified domain. The number includes all processes in all states.

As with other functions, you can pass a null domain handle to imply "the current domain."

See Also: X_PTerm, X_PCreate, Z_PHandleListGet

Z_DomRootProcessGet (2104H)

Function

Get the handle of the root process in a specified domain.

Entry Parameters

Register	Value
CX	**2104H**
EDX	**Domain handle (0=current domain)**

Returned Values

Register	Value	Meaning
EBX	**Process Handle**	
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	**Invalid handle**

Explanation

Z_DomRootProcessGet returns the process handle of the root process in the specified domain. The root process is the process that was created when the domain was first created. All other processes in the same domain are its children or their descendents.

As with other functions, you can pass a null domain handle to imply "the current domain."

See Also: X_PCreate, X_DomNProcessesGet

Z_DomSuspend (1304H)

Function

Suspend all processes in a specified domain.

Entry Parameters

Register	Value
CX	**1304H**
EDX	**Domain handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	 **Invalid handle**

Explanation

Z_DomSuspend calls X_PSuspend for each process in the specified domain. See X_PSuspend on page 2-140 for details. You can unsuspend all the suspended processes in a domain by calling Z_DomUnsuspend.

See Also: X_PSuspend, Z_DomUnsuspend

Z_DomTerm (1604H)

Function

Terminate all processes in the specified domain.

Entry Parameters

Register	Value
CX	**1604H**
EDX	**Domain handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	 **Invalid handle**

Explanation

Z_DomTerm calls X_PTerm for each process in the specified domain. This terminates the processes.

See Also: X_PTerm

Z_DomUnsuspend (1404H)

Function

Resume execution of all processes in a specified domain.

Entry Parameters

Register	Value
CX	**1404H**
EDX	**Domain handle (0=current domain)**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **36H - E_BadDomain**	 **Invalid handle**

Explanation

Z_DomUnsuspend resumes execution of all the suspended processes in a domain. The processes may have been suspended by calls to X_PSuspend or Z_DomSuspend.

See Also: Z_DomSuspend, X_PSuspend

Z_EndOfInterrupt (1D04H)

Function

Signal end of interrupt.

Entry Parameters

Register	Value
CX	**1D04H**
EDX	**Interrupt being cleared (0 to 0FH)**

Returned Values

Register	Value	
ECX	**Error code:** **00H - No error**	

Explanation

Z_EndOfInterrupt must be called from level 0 hardware interrupt handlers which clear the In-Service bit of the Programmable Interrupt Controller (PIC).

See Also: Z_HandlerHWInt

Z_FlagAlloc (1C02H)

Function

Allocate a flag.

Entry Parameters

Register	Value
CX	**1C02H**

Returned Values

Register	Value	Meaning
EBX	**Flag number**	
ECX	**Error code:** **00H - No error** **2DH - E_NoFlag**	 **All flags allocated**

Explanation

Z_FlagAlloc allocates a flag, sets its initial state to Off, and returns the flag number. Some system flags are preallocated, but other flags must all be allocated from the pool.

There is a very limited number of flags available (normally 256), so use them sparingly.

When you no longer need a flag, you can free it by calling Z_FlagFree.

See Also: Z_FlagFree, X_FlagsMaxGet, Z_FlagSet, Z_FlagStatusGet, Z_FlagWait

Z_FlagFree (1D02H)

Function

Release a flag.

Entry Parameters

Register	Value
CX	**1D02H**
EDX	**Flag number**

Returned Values

Register	Value	Meaning
EBX	**Flag number**	
ECX	**Error code:** **00H - No error** **04H - E_BadFlag** **2EH - E_FlagBusy**	 **No such flag** **Flag in use**

Explanation

Z_FlagFree releases a flag, freeing it to be allocated to some other user.

Z_FlagFree will not free a flag that is set, or on which a process is waiting. You can call Z_FlagStatusGet to determine the flag's current status.

See Also: Z_FlagAlloc, X_FlagsMaxGet, Z_FlagSet, Z_FlagStatusGet, Z_FlagWait

Z_FlagSet (0302H)

Function

Set a flag.

Entry Parameters

Register	Value
CX	**0302H**
EDX	**Flag number**

Returned Values

Register	Value	Meaning
EBX	**Flag number**	
ECX	**Error code:** **00H - No error** **04H - E_BadFlag** **05H - E_FlagOverrun**	 **Invalid flag number** **Overrun detected**

Explanation

Z_FlagSet sets a flag to indicate that a process waiting on the flag should immediately become ready to run.

If the flag was already set, the overrun counter is incremented and the error code E_FlagOverrun is returned to the ISR. The same code will be returned to the process that eventually calls Z_FlagWait.

See Also: Z_FlagStatusGet, Z_FlagWait, Z_FlagWWTO

Z_FlagStatusGet (1902H)

Function

Return the status of a flag.

Entry Parameters

Register	Value
CX	**1902H**
EDX	**Flag number**

Returned Values

Register	Value	Meaning
EBX	**Status:** **00H - Unused** **01H - Flag_Off** **02H - Flag_On** **>2 - Process handle**	 **Unallocated** **Unset** **Set** **Unset, process waiting**
ECX	**Error code:** **00H - No error** **04H - E_BadFlag**	 **Flag number too big**

Explanation

Z_FlagStatusGet returns the status of a specified flag. If the returned value is above 2 then there is a process waiting on the flag and the returned value is its handle.

See Also: Z_FlagAlloc, Z_FlagFree, X_FlagsMaxGet, Z_FlagSet, Z_FlagWait

Z_FlagWait (0202H)

Function

Wait for a flag.

Entry Parameters

Register	Value
CX	**0202H**
EDX	**Flag number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **04H - E_BadFlag** **05H - E_FlagOverrun** **06H - E_FlagUnderrun**	 **Flag number too big** **Flag has been set twice** **Someone else is waiting**

Explanation

Z_FlagWait makes the calling process wait indefinitely until the specified flag has been set, unless it is already set, in which case the calling process can continue. Use the related function Z_FlagWWTO if you want to specify a timeout.

If, at the time of this call, the flag has been set two or more times, the error code E_FlagOverrun is returned. If another process is waiting on the flag, the code E_FlagUnderrun is returned.

The act of calling Z_FlagWait or Z_FlagWWTO resets the flag or decrements the overrun counter. If overrun has occurred, you must call Z_FlagWait or Z_FlagWWTO once for each Z_FlagSet there has been.

See Also: Z_FlagSet, Z_FlagWWTO

Z_FlagWWTO (2002H)

Function

Wait, with a timeout, for a flag.

Entry Parameters

Register	Value
EBX	**Timeout count in ticks**
CX	**2002H**
EDX	**Flag number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **04H - E_BadFlag** **05H - E_FlagOverrun** **06H - E_FlagUnderrun** **2FH - E_FlagTimeout**	 **Flag number too big** **Flag had been set twice** **Someone else is waiting** **Timeout. Flag not set**

Explanation

Z_FlagWWTO functions exactly as Z_FlagWait, except that if the flag remains unset after the specified number of clock ticks, then the process is restarted with the error code E_FlagTimeout. Timeouts from 0 to 65535 ticks may be specified.

If, at the time of this call, the flag has been set two or more times, the error code E_FlagOverrun is returned. If another process is waiting on the flag, the code E_FlagUnderrun is returned.

See Also: Z_FlagSet, Z_FlagWait

X_FlagsMaxGet (1E02H)

Function

Return the number of flags in the system.

Entry Parameters

Register	Value
CX	**1E02H**

Returned Values

Register	Value	Meaning
EBX	**Number of flags**	
ECX	**Error code:** **00H - No error**	 **No error is possible**

Explanation

X_FlagsMaxGet returns the number of flags the system supports.

See Also: Z_FlagAlloc, Z_FlagFree, Z_FlagStatusGet

X_ForeCheck (0801H)

Function

Check if in the foreground.

Entry Parameters

Register	Value
CX	**0801H**

Returned Values

Register	Value	Meaning
EBX	**00H** **xxH**	**Background** **Foreground**
ECX	**Error code:** **00H - No error**	**No error is possible**

Explanation

Call X_ForeCheck if your application needs to know if it is running in the foreground. Only one domain is in the foreground at any instant. It owns the physical screen and is attached to the keyboard and mouse.

The user tells the task manager which task to bring to the foreground. The task manager in turn commands the VM and DOM modules to switch the selected domain into the foreground.

While running in the background, applications can read and write video memory and controllers (up to VGA resolution only), but the user will see nothing until they switch your application into the foreground.

If you wait for a keypress or mouse activity while in background, your application will be suspended until it is in the foreground again.

If, while running in the background, you urgently need to display a message to the user, you can call VM functions to do so. You cannot force your domain to the foreground, but you could ask the user to do so by pressing the appropriate task manager hot key combination.

See Also: X_ForeGet

X_ForeGet (0B05H)

Function

Get the console owner's domain handle.

Entry Parameters

Register	Value
CX	**0B05H**

Returned Values

Register	Value	Meaning
EBX	**Handle**	**Domain handle of console owner**
ECX	**Error code:** **00H - No error**	

Explanation

X_ForeGet gets the domain handle of the console owner, in other words the foreground domain. Just as ownership is switching, the console may have no owner, so the handle returned may be zero.

See Also: X_ForeCheck

Z_GateAlloc (2F03H)

Function

Allocate a call gate.

Entry Parameters

Register	Value
CX	**2F03H**
DS:EDX	**Pointer to a Gate Parameter Block; see "Explanation"**

Returned Values

Register	Value	Meaning
EBX	**=GATE_PB_GATE**	**Gate selector**
ECX	**Error code:** **00H - No error** **31H - E_BadDesc** **32H - E_NoDesc**	 **Invalid parameters** **No free descriptors**
DS:[EDX]	**GATE_PB_GATE**	**Gate selector**

Explanation

Z_GateAlloc allocates a protected mode call gate and returns its selector.

The parameter block is defined in STRUCS.DEF as follows:

GATE_PBLK	struc	
GATE_PB_SEL	dw	?
GATE_PB_COUNT	dw	?
GATE_PB_OFFSET	dd	?
GATE_PB_GATE	dw	?
unused	db	?
GATE_PB_SINFO	db	?
GATE_PBLK	ends	

Before you call Z_GateAlloc, you must initialize all fields in GATE_PBLK except GATE_PB_GATE, which will be filled with the selector of the allocated gate when the function returns successfully.

GATE_PB_SEL and GATE_PB_OFFSET are the address of the destination procedure. GATE_PB_COUNT should be 0.

GATE_PB_SINFO is an image of the byte within the gate which holds its DPL. Only bits 5 and 6 are used:

Bit	Reset	Set
5, 6	Privilege level	Privilege level

Note that this function creates a call gate, not a task, interrupt or trap gate. Call Z_GateFree to deallocate the gate when it is no longer needed.

See Also: Z_GateFree

Z_GateFree (3003H)

Function

Release a call gate.

Entry Parameters

Register	Value
CX	**3003H**
DS:EDX	**Pointer to a Gate Parameter Block; see "Z_GateAlloc (2F03H)" on page 2-50**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **31H - E_BadDesc**	 **Invalid selector**

Explanation

Z_GateFree releases a call gate. The parameter block is as described under Z_GateAlloc. Only the GATE_PB_GATE field need be filled in when you call Z_GateFree.

See Also: Z_GateAlloc

Z_HandlerGenEx (0704H)

Function

Install a general exception handler.

Entry Parameters

Register	Value
CX	**0704H**
DS:EDX	**Pointer to Handler Parameter Block; see "Explanantion"**

Returned Values

Register	Value	Meaning
ECX	**Error code: 00H - No error**	
DS:[EDX]	**Modified**	**See "Explanation"**

Explanation

Z_HandlerGenEx installs a handler for General Exceptions. The following description applies in most respects also to the following functions:

Z_HandlerHWInt

Z_HandlerHWIntDflt

Z_HandlerIOEx

Z_HandlerPageFault

Z_HandlerSWInt

Z_HandlerVHWInt

Z_HandlerVHWIntDflt

Differences between the functions are explained in the individual function descriptions.

Handlers are themselves level zero protected mode code functions. Before you install a handler, you must obtain a code descriptor and selector for its code segment.

In each case, the function to install the handler takes as a parameter a pointer to the following Handler Parameter Block:

HPB	struc	

	dw	0FFFFH
	dw	0
HPB_EIP	dd	?
HPB_CS	dd	?
	dd	4 dup (?)
HPB	ends	

Before you call the function to install the handler, you must supply the HPB_EIP and HPB_CS fields, which should point to the handler itself.

The last handler in each chain is the default handler provided by the operating system. Handlers are installed just before this default handler, so the first installed handler always remains first on the chain. When an exception occurs, the handlers are called in order from the first on the chain to the last.

All exceptions and software interrupts (but not hardware interrupts) are considered to belong to the current domain. Each domain therefore has its own chains of handlers for exceptions and software interrupts.

You can uninstall any handler by calling Z_HandlerUnlink.

Handler Interface

All handlers are called with the same stack frame, and handler-specific information in the registers. This information is detailed in the sections describing each handler installation function.

In each case, DS:EBX contains the address of the Handler Parameter Block passed when this handler was installed, so it can find its data if it needs to. Note that the memory above or below this HPB can be used to store data that may be useful to the handler.

The stack frame on entry to the handler includes the following structure, defined in STRUCS.DEF, with SS:EBP pointing to its base:

EXCEP	struc	
EX_EDI	dd	?
EX_ESI	dd	?
EX_EBP	dd	?
EX_OPCODE	db	?,0,0,0
EX_EBX	dd	?
EX_EDX	dd	?
EX_ECX	dd	?

EX_EAX	dd	?
EX_ERROR	dd	?
EX_EIP	dd	?
EX_CS	dw	?
EX_OSINFO	dw	?
EX_EFLAGS	dd	?
EX_ESP	dd	?
EX_SS	dw	?,?
EX_ES	dw	?,?
EX_DS	dw	?,?
EX_FS	dw	?,?
EX_GS	dw	?,?
EXCEP	ends	

EX_OPCODE contains flags that will be set if the instruction had any prefixes, or if the address size or operand size is 32 bits. STRUCS.DEF provides equates for all the used bits.

Bit	**Equated name**	**Use if set:**
0		Not used
1	EXOP_ADDR	AddressSize=32 (override xor 32-bit CS)
2	EXOP_SIZE	OperandSize=32 (override xor 32-bit CS)
3		Not used
4-6	EXOP_SEG EXOP_CS EXOP_SS EXOP_DS EXOP_ES EXOP_FS EXOP_GS	Segment override. EXOP_SEG=70H, CS=10H, SS=20H, DS=30H, ES=40H, FS=50H, GS=60H
7	EXOP_REP	Repeat prefix (REP/REPE or REPNE)

EX_OSINFO contains the 386 exception number in its low byte, and flags in its high byte.

Bit	**Use if set:**
0-7	386 exception number
8	Processor exception

9	Hardware interrupt
10	Software interrupt

The system disables interrupts before calling the handler and restores the previous interrupt state after the handler returns.

General Exception Handler Entry Parameters

General exception handlers are passed the following information in registers:

Register	Value
DS:EBX	Pointer to Handler Parameter Block
DL	Exception number

Return from the Handler

Handlers must return to the address on the stack by executing a 32-bit RETF. If the handler's code segment is 16-bit, it can execute a 32-bit RETF by placing an operand override byte (66H) in front of the RETF instruction.

To indicate success in handling the exception or interrupt, all handlers must clear the carry flag (CF) and place their return code in AX. The system will not call any other handlers in the chain.

To indicate failure, set CF. The system will call the next handler in the chain.

DX must be preserved by some handlers, and set by others, as detailed in the descriptions that follow of each handler installation function.

General Exception Handler Exit Codes

General exception handlers must return the following in register AX if they have handled the exception (CF clear):

Register	Value	Meaning
AX	0 0FFFEH 0FFFFH	Success: return to user Suspend user Terminate user

The default General Exception handler reflects exceptions 0 to 5 and 7 to real mode, and terminates the user process for other exceptions.

See Also: Z_HandlerHWInt, Z_HandlerHWIntDflt, Z_HandlerIOEx, Z_HandlerPageFault, Z_HandlerSWInt, Z_HandlerUnlink, Z_HandlerVHWInt, Z_HandlerVHWIntDflt

Z_HandlerHWInt (0804H)

Function

Install a level 0 hardware interrupt handler.

Entry Parameters

Register	Value
CX	**0804H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	Meaning
ECX	**Error code: 00H - No error**	
DS:[EDX]	**Modified**	**As for Z_HandlerGenEx**

Explanation

Z_HandlerHWInt installs a level 0 handler for hardware interrupts. Such handlers are called before the system itself processes the interrupt. You can also install a virtual interrupt handler by calling Z_HandlerVHWInt, or replace the default handler by calling Z_HandlerHWIntDflt. Virtual hardware interrupt handlers are invoked, after switching to a specified domain, if the level 0 interrupt handler chain indicates that the interrupt is to be reflected.

Most details are the same as for Z_HandlerGenEx (see page 2-53) except for the following details of the handler's entry and exit conditions.

Handler Entry Parameters

Register	Value
DS:EBX	**Pointer to Handler Parameter Block**
DL	**Interrupt number (must be preserved)**
DH	**EX_OSINFO (Must be preserved)**

EX_OSINFO is described under Z_HandlerGenEx.

Handler Exit

If the interrupt has not been handled, set the carry flag (CF) to pass the interrupt on, otherwise clear it and return the following:

Register	Value	Meaning
CF	Clear	Interrupt handled
AL	0 1	Return from interrupt Reflect it
AH	0 to 0FH 0FFH	Interrupt level Interrupt completed
EBX	0 Domain handle 0FFFFFFFFH	No domain switch Domain to switch to Switch to next domain
DX	Unchanged	

If a level 0 hardware interrupt handler needs to signal End of Interrupt (EOI) by clearing the PIC's In-Service bit, it must inform the system of this by calling Z_EndOfInterrupt.

See Also: Z_EndOfInterrupt, Z_HandlerGenEx, Z_HandlerUnlink

Z_HandlerHWIntDflt (1804H)

Function

Install a substitute for the default level 0 hardware interrupt handler.

Entry Parameters

Register	Value
ECX	1804H
DS:[EDX]	Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53

Returned Values

Register	Value	Meaning
CX	Error code: 00H - No error	
[EDX]	Modified	As for Z_HandlerGenEx

Explanation

Z_HandlerHWIntDflt installs a substitute for the default level 0 handler for hardware interrupts. The default handler is called only if no installable handler has handled the interrupt.

Details are the same as for Z_HandlerHWInt (see page 2-59), except that the last level 0 hardware interrupt handler must not pass the interrupt on; it must clear CF to return to the system.

See Also: Z_EndOfInterrupt, Z_HandlerGenEx, Z_HandlerHWInt, Z_HandlerUnlink

Z_HandlerIOEx (0604H)

Function

Install an I/O exception handler.

Entry Parameters

Register	Value
CX	0604H
DS:EDX	Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error**	
DS:[EDX]	**Modified**	**As for Z_HandlerGenEx**

Explanation

Z_HandlerIOEx installs a handler for I/O Exceptions. When a General Exception occurs, the system decodes the offending instruction and calls the I/O exception handlers if the instruction is IN, INS, OUT or OUTS. Details are the same as for Z_HandlerGenEx (see page 2-53), except for the following details of the handler entry and exit conditions.

Handler Entry Parameters

Register	Value
AL	**Value to be output, if OUT <port>,AL instruction**
AH	**Access type. See below**
DS:EBX	**Pointer to Handler Parameter Block**
DX	**Port number**

The Access Type in AH indicates whether this is an input or output operation, whether 8, 16 or 32-bit, if it is a string operation, and if it has a repeat prefix.

AL contains the value to be output only if the instruction is a simple 8-bit OUT instruction. For all other OUT and OUTS instructions you must determine the value(s) to be output from the image of the application's registers in the stack frame. The stack frame is described under Z_HandlerGenEx.

AH contains the Access Type flags (which are similar to, but not identical to, the EX_OPCODE field in the stack frame). STRUCS.DEF provides equates for all the used bits.

Bit	Equated name	Reset	Set
0	IO_EX_OUT	IN or INS	OUT or OUTS
1	IO_EX_ADDR	AddressSize=16	AddressSize=32
2	IO_EX_SIZE	OperandSize=16	OperandSize=32
3	IO_EX_WORD	Byte I/O	Word or Dword I/O
4-6	IO_EX_STRING	IN or OUT	INS or OUTS
7	IO_EX_REP		REP

To test for a string operation, you should use IO_EX_STRING (70H) as a mask and treat any non-zero value as a string operation, although any value other than 70H would indicate a segment override prefix. String operations use EDI/ESI if IO_EX_ADDR is set, otherwise DI/SI. The values for these registers can be read from the stack frame (for example EX_EDI[ebp]), as can the segment registers to be used. The segment register values in the stack frame are protected mode selectors that point to the actual application's segments.

String operations may have a repeat prefix, indicated by IO_EX_REP being set, in which case you must read the stack frame for the count and the direction flag (In EX_EFLAGS).

If IO_EX_WORD is reset, the operation is byte I/O. If IO_EX_WORD is set, IO_EX_SIZE distinguishes between a word and a dword operation.

Handler Exit

If the exception has not been handled, set the carry flag to pass it on, otherwise clear it and return the following:

Register	Value	Meaning
AX	0 to 0FFH 0FFFEH 0FFFFH	Success: see below Suspend user Terminate user

If the faulting instruction was an IN AL,DX or IN AL,imm8 return the input value in AL. For all other IN instructions write the input value into the stack frame (EX_AX or EX_EAX) or into memory at ES:(E)DI for INS.

See Also: Z_HandlerGenEx, Z_HandlerUnlink

NOTE: It is normally sufficient to handle the cases where AH=0 (byte input) or AH=1 (byte output) only. All other cases can be passed to the default handler, which will break up the more complicated cases into repeated calls using AH=0 or 1 as appropriate. The string and other cases need be handled only in special circumstances (where speed is critical, for example).

Z_HandlerPCreate (1F04H)

Function

Install a process create handler.

Entry Parameters

Register	Value
CX	**1F04H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	
ECX	**Error code:** **00H - No error**	

Explanation

Z_HandlerPCreate installs a global handler that is called twice every time a process is created. Z_HandlerPTerm installs a handler to be called when processes are deleted. The installation process is the same as for Z_HandlerGenEx (see page 2-53).

Handlers are level zero protected mode code functions. Before you install a handler, you must obtain a code descriptor and selector for its code segment.

When a process is being created, the handlers are called in order from the first installed to the last. The handlers are called in the context of the domain of the process being created.

You can uninstall a handler by calling Z_HandlerUnlink.

Handler Interface

Handlers are called with the stack frame shown below, and with DS:EBX set to the address of the Handler Parameter Block passed when this handler was installed. Note that the memory above or below this HPB can be used to store data that may be useful to the handler.

The stack frame on entry to the handler includes a PCREATE_PBLK as described under X_PCreate on page 2-126. The CALL_PB_RSRVD field contains the parent domain's handle. Other fields are as described under X_PCreate.

Process Create handlers are passed the following information in registers:

Register	Value
DS:EBX	Pointer to Handler Parameter Block
CX	Create phase: 0 - First phase (initialize) 1 - Second phase (startup)

SS:EBP	Pointer to PCREATE_PBLK

The handler is called in the context of the new process' domain. You can check if it is the first process in a new domain by calling X_DomNProcessesGet.

The system calls the handler twice for each new process. During the first phase, if it is a new domain, handlers should initialize the domain so that they are capable of providing any services required by other modules (such as exception handlers), but they should not do anything that depends on the services of other modules (such as exception handlers).

During the second phase, you can assume that all other handlers have completed their first phase.

If a handler cannot initialize, it must call X_PTerm. This will not return.

Return from the Handler

Handlers must pass control to the next handler in the chain by setting the CY flag and executing a 32-bit RETF.

See Also: Z_HandlerGenEx, Z_HandlerPTerm, Z_HandlerUnlink

Z_HandlerPTerm (2004H)

Function

Install a process termination handler.

Entry Parameters

Register	Value
CX	**2004H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	.
ECX	**Error code:** **00H - No error**	

Explanation

Z_HandlerPTerm installs a global handler that is called twice every time a process is terminated. The installation process is the same as for Z_HandlerGenEx (see page 2-53).

Handlers are level zero protected mode code functions. Before you install a handler, you must obtain a code descriptor and selector for its code segment.

When a process is being terminated, the handlers are called in order from the first installed to the last. The handlers are called in the context of the domain of the process being terminated.

You can uninstall a handler by calling Z_HandlerUnlink.

Handler Interface

Handlers are called with DS:EBX set to the address of the Handler Parameter Block passed when this handler was installed. Note that the memory above or below this HPB can be used to store data that may be useful to the handler.

Process Termination handlers are passed the following information in registers:

Register	Value
DS:EBX	Pointer to Handler Parameter Block
CX	Termination phase: 0 - First phase (stop) 1 - Second phase (remove)

The handler is called in the context of the process being terminated. You can check if it is the last process in a domain by calling X_DomNProcessesGet.

The system calls the handler twice for each termination. During the first phase, if it is the last process in a domain, handlers should stop doing anything dependent on the services of other modules (such as exception handlers). During the second phase, you can assume that all other handlers have completed their first phase and you should free up any resources that are no longer needed.

When a domain is terminated the system frees up the following:

- Local memory allocated by Z_MemAlloc
- All instance data and descriptors
- Instanced memory

Return from the Handler

Handlers must pass control to the next handler in the chain by setting the CY flag and executing a 32-bit RETF.

See Also: Z_HandlerGenEx, Z_HandlerPCreate, Z_HandlerUnlink

Z_HandlerPageFault (0504H)

Function

Install a page fault handler.

Entry Parameters

Register	Value
CX	**0504H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	Meaning
ECX	**Error code: 00H - No error**	
DS:[EDX]	**Modified**	**As for Z_HandlerGenEx**

Explanation

Z_HandlerPageFault installs a handler for Page Faults. Most details are the same as for Z_HandlerGenEx (see page 2-53) except for the following details of the handler entry and exit conditions.

Handler Entry Parameters

Register	Value
CR2	**Faulting address**
DS:EBX	**Pointer to Handler Parameter Block**

Handler Exit

If the fault has not been handled, set the carry flag to pass it on, otherwise clear it and return the following:

Register	Value	Meaning
AX	0 0FFFEH 0FFFFH	Success: return to user Suspend user Terminate user

If the exception is to be passed on, preserve CR2 and return with interrupts disabled.

If the exception is to be reflected to the real mode Interrupt 14 handler, it should be passed to the next handler in the chain.

Note that if the handler wants to enable interrupts it should first save the contents of CR2.

See Also: Z_HandlerGenEx, Z_HandlerUnlink

Z_HandlerSWInt (0404H)

Function

Install a software interrupt handler.

Entry Parameters

Register	Value
CX	**0404H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	Meaning
ECX	**Error code: 00H - No error**	
DS:[EDX]	**Modified**	**As for Z_HandlerGenEx**

Explanation

Z_HandlerSWInt installs a handler for software interrupts. Most details are the same as for Z_HandlerGenEx (see page 2-53) except for the following details of the handler entry and exit conditions.

Handler Entry Parameters

Register	Value
DL	**Interrupt number**
DS:EBX	**Pointer to Handler Parameter Block**

Handler Exit

If the interrupt has not been handled, set the carry flag to pass it on, otherwise clear it and return the following:

Register	Value	Meaning
AX	0	Return from interrupt

If the interrupt is to be reflected to real mode, pass it on. The default handler will do the reflection.

If you handle the interrupt, you may return values in the user's register image in the stack frame.

See Also: Z_HandlerGenEx

Z_HandlerSwapIn (1B04H)

Function

Install a dispatcher swap-in handler.

Entry Parameters

Register	Value
CX	**1B04H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error**	
DS:[EDX]	**Modified**	**As for Z_HandlerGenEx**

Explanation

Z_HandlerSwapIn installs a global handler that is called every time a process is dispatched. The installation process is the same as for Z_HandlerGenEx (see page 2-53).

Handlers are themselves protected mode code functions. Before you install a handler, you must obtain a code descriptor and selector for its code segment.

When a process is being dispatched, interrupts are off, so any code you hook in will affect the system's interrupt latency time. If possible, avoid installing a handler at all. The handlers are called in the context of the process being dispatched in.

You can uninstall a handler by calling Z_HandlerUnlink.

Handler Interface

Swap-in handlers are passed the following information in registers:

Register	Value
DS:EBX	Pointer to Handler Parameter Block

Note that the memory above or below this HPB can be used to store data that may be useful to the handler.

Return from the Handler

Handlers must pass control to the next handler in the chain by setting the carry flag and executing a 32-bit RETF.

See Also: Z_HandlerGenEx, Z_HandlerSwapOut, Z_HandlerUnlink

Z_HandlerSwapOut (1C04H)

Function

Install a dispatcher swap-out handler.

Entry Parameters

Register	Value
CX	**1C04H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	Meaning
ECX	**Error code: 00H - No error**	
DS:[EDX]	**Modified**	**As for Z_HandlerGenEx**

Explanation

Z_HandlerSwapOut installs a global handler that is called every time a process is dispatched out. The installation process is the same as for Z_HandlerGenEx (see page 2-53).

Handlers are themselves protected mode code functions. Before you install a handler, you must obtain a code descriptor and selector for its code segment.

When a process is being dispatched, interrupts are off, so any code you hook in will affect the system's interrupt latency time. If possible, avoid installing a handler at all. The handlers are called in the context of the process being dispatched out.

You can uninstall a handler by calling Z_HandlerUnlink.

Handler Interface

Swap Out handlers are passed the following information in registers:

Register	Value
DS:EBX	Pointer to Handler Parameter Block

Note that the memory above or below this HPB can be used to store data that may be useful to the handler.

Return from the Handler

Handlers must pass control to the next handler in the chain by setting the carry flag and executing a 32-bit RETF.

See Also: Z_HandlerGenEx, Z_HandlerSwapIn, Z_HandlerUnlink

Z_HandlerUnlink (1704H)

Function

Uninstall a dispatch, exception, interrupt or process creation handler.

Entry Parameters

Register	Value
CX	**1704H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **31H - E_BadDesc**	 **No PREV handler in chain**

Explanation

Z_HandlerUnlink unhooks an installed handler from its chain. The Handler Parameter Block you pass must be the same one passed in the corresponding call to install the handler.

See Also: Z_HandlerGenEx, Z_HandlerPCreate, Z_HandlerSwapIn

Z_HandlerVHWInt (1904H)

Function

Install a virtual hardware interrupt handler.

Entry Parameters

Register	Value
CX	**1904H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	Meaning
ECX	**Error code: 00H - No error**	
DS:[EDX]	**Modified**	**As for Z_HandlerGenEx**

Explanation

Z_HandlerVHWInt installs a virtual handler for hardware interrupts. Virtual hardware interrupt handlers are invoked, after switching to a specified domain, if the level 0 interrupt handler chain indicates that the interrupt is to be reflected.

Details are the same as for Z_HandlerGenEx (see page 2-53), except for the following details of the handler entry and exit conditions.

Handler Entry Parameters

Register	Value
DS:EBX	**Pointer to Handler Parameter Block**
DL	**Interrupt number (must be preserved)**
DH	**EX_OSINFO (Must be preserved)**

EX_OSINFO is described under Z_HandlerGenEx.

Handler Exit

If the interrupt has not been handled, set the carry flag to pass the interrupt on, otherwise clear it and return the following:

Register	Value	Meaning
CF	Clear	Interrupt handled
AL		

	0 1	Return from interrupt Reflect to real mode
AH	0 to 0FH 0FFH	Interrupt level Interrupt completed
DX	Unchanged	

See Also: Z_EndOfInterrupt, Z_HandlerGenEx, Z_HandlerUnlink

Z_HandlerVHWIntDflt (1A04H)

Function

Install a substitute for the default virtual hardware interrupt handler.

Entry Parameters

Register	Value
CX	**1A04H**
DS:EDX	**Pointer to Handler Parameter Block; see "Z_HandlerGenEx (0704H)" on page 2-53**

Returned Values

Register	Value	Meaning
ECX	**Error code: 00H - No error**	
DS:[EDX]	**Modified**	**As for Z_HandlerGenEx**

Explanation

Z_HandlerVHWIntDflt installs a substitute for the default virtual handler for hardware interrupts. Virtual hardware interrupt handlers are invoked, after switching to a specified domain, if the level 0 interrupt handler chain indicates that the interrupt is to be reflected to real mode. The default virtual handler is invoked at the end of the chain of installed virtual interrupt handlers, if there is one.

Most details are the same as for Z_HandlerGenEx (see page 2-53), except for the following details of the handler's entry and exit conditions.

Handler Entry Parameters

Register	Value
DS:EBX	**Pointer to Handler Parameter Block**
DL	**Interrupt number**
DH	**EX_OSINFO (Must be preserved)**

EX_OSINFO is described under Z_HandlerGenEx.

Handler Exit

The default handler must return the following:

Register	Value	Meaning
CF	Clear	Interrupt handled
AL	0 1	Return from interrupt Reflect to real mode
AH	0 to 0FH 0FFH	Interrupt level Interrupt completed
DX	Unchanged	

See Also: Z_EndOfInterrupt, Z_HandlerGenEx, Z_HandlerUnlink

Z_IOBitmapDomGet (0B04H)

Function

Read I/O bitmap of a specified domain.

Entry Parameters

Register	Value
CX	**0B04H**
DS:EDX	**Pointer to a BANKIOMAP_PBLK; see "Explanation"**

Returned Values

Register	Value	Meaning
EBX	**0** **1**	**Bitmap bit reset** **Bitmap bit set**
ECX	**Error code:** **00H - No error**	

Explanation

Z_IOBitmapDomGet reads the I/O bitmap of the specified domain. Thus it is similar to Z_IOBitmapGet except that it reads the bitmap of a different domain.

The parameter block is defined as follows:

BANKIOMAP_PBLK	struc	
BANKIO_PB_DOMAIN	dd	?
BANKIO_PB_PORT	dw	?
BANKIO_PB_BITVAL	dw	?
BANKIOMAP_PBLK	ends	

You must supply the BANKIO_PB_DOMAIN and BANKIO_PB_PORT values.

On return, BX and BANKIO_PB_BITVAL will contain 0 or 1. If the port number is illegally high, 1 is returned.

See Also: Z_IOBitmapSet, Z_IOBitmapDomSet

Z_IOBitmapDomSet (0C04H)

Function

Set or reset a bit in the I/O bitmap of a specified domain.

Entry Parameters

Register	Value
EBX	**0 - reset** **1 - set**
CX	**0C04H**
DS:EDX	**Pointer to a BANKIOMAP_PBLK; see "Explanation"**

Returned Values

Register	Value	Meaning
EBX	**0** **1**	**Bitmap bit was reset** **Bitmap bit was set**
ECX	**Error code:** **00H - No error**	

Explanation

Z_IOBitmapDomSet sets or resets the I/O bitmap bit corresponding to the specified port in the bitmap of the specified domain. Thus it is similar to Z_IOBitmapSet except that it affects a different domain.

Input from or output to a port whose bit is set will cause an I/O exception that can be trapped by a handler you install by calling Z_HandlerIOEx.

The parameter is a pointer to a BANKIOMAP_PBLK, as described under Z_IOBitmapDomGet. You must supply all three fields before calling Z_IOBitmapDomSet.

The previous value of the bit is returned in BX and BANKIO_PB_BITVAL. If the port number is illegally high, BX=1 is returned.

See Also: Z_HandlerIOEx, Z_IOBitmapGet, Z_IOBitmapDomSet

Z_IOBitmapGet (0904H)

Function

Read I/O bitmap.

Entry Parameters

Register	Value
CX	**0904H**
EDX	**Port number**

Returned Values

Register	Value	Meaning
EBX	**0** **1**	**Bitmap bit reset** **Bitmap bit set**
ECX	**Error code:** **00H - No error**	

Explanation

Z_IOBitmapGet reads the current domain's I/O bitmap. If the port number is illegally high, BX=1 is returned.

See Also: Z_IOBitmapSet, Z_IOBitmapDomGet

Z_IOBitmapSet (0A04H)

Function

Set or reset a bit in the I/O bitmap.

Entry Parameters

Register	Value
EBX	**0 - reset** **1 - set**
CX	**0A04H**
EDX	**Port number**

Returned Values

Register	Value	Meaning
EBX	**0** **1**	**Bitmap bit was reset** **Bitmap bit was set**
ECX	**Error code:** **00H - No error**	

Explanation

Z_IOBitmapSet sets or resets the current domain's I/O bitmap bit corresponding to the specified port. If any application does an I/O operation to a port whose bit is set, an exception is generated, as described in the Intel Programmer's Reference Guides. Initially all bits are reset except for those that VM virtualizes.

Input from, or output to, a port whose bit is set will cause an I/O exception that can be trapped by a handler you install by calling Z_HandlerIOEx.The previous value of the bit is returned in BX. If the port number is illegally high, BX=1 is returned.

See Also: Z_HandlerIOEx, Z_IOBitmapDomSet, Z_IOBitmapGet

Z_InstanceSet (1104H)

Function

Set the map of instanced memory addresses.

Entry Parameters

Register	Value
CX	**1104H**
DS:EDX	**Pointer to an INSTANCE_REGISTRATION structure; see "Explanation"**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **03H - E_NoMemory**	 **Not enough memory**

Explanation

Z_InstanceSet discards the old map, and creates a completely new map of memory areas to be instanced. It must only be called as part of the initialization of the task manager.

The parameter is a pointer to an INSTANCE_REGISTRATION structure, which is defined in STRUCS.DEF as follows

INSTANCE_REGISTRATION	struc	
IR_PAGES	dw	64 dup (?)
IR_GLOBAL	dd	?
IR_INSTANCE	dd	?
INSTANCE_REGISTRATION	ends	

:

IR_PAGES is a bitmap of 1024 bits, where each bit represents the state of a 4 KB page within the first 4 MB of the linear address space. Bit 0 of the first byte corresponds to page 0. A bit is set if the corresponding page is instanced. A page should be instanced if significantly more than half of its 4 KB address space is instanced. Any spaces within it that are global must then be specified in SWAPLISTs in the IR_GLOBAL list described below.

If less than half a given page is instanced, the page's bit in IR_PAGES should be reset, so that the page will be treated as global. Any spaces within it that are instanced must then be specified in SWAPLISTs in the IR_INSTANCE list. Also, any instanced addresses above 4 MB must be specified in this list.

There is a tradeoff between performance and memory use in the decision about whether to use an instance page when more than half, but not all, the page consists of instanced addresses. This is because domain switches involve

- Copy addresses specified in IR_INSTANCE into Domain A's buffer
- Copy addresses specified in IR_GLOBAL into the global memory buffer
- Change from Domain A's to Domain B's base page table
- Copy addresses specified in IR_GLOBAL from the global memory buffer
- Copy addresses specified in IR_INSTANCE from Domain B's buffer Each domain has a buffer equal to the sum of the SL_COUNTs of IR_INSTANCE, and has a 4 KB page per set bit in IR_PAGES.

  IR_GLOBALand IR_INSTANCE are pointers to arrays of contiguous SWAPLISTs. Each SWAPLIST specifies a single block of memory addresses as being global or instanced respectively. The two arrays of SWAPLISTs are each terminated by a SWAPLIST with an SL_COUNT field of zero.

  The SWAPLIST structure is defined in STRUCS.DEF as follows

SWAPLIST	struc	

SL_COUNT	dw	?
SL_BASE	dd	?
SWAPLIST	ends	

:

SL_COUNT is the number of contiguous bytes in the item.

SL_BASE is the linear base address of the item.

X_MXCreate (3702H)

Function

Allocate a mutex zone.

Entry Parameters

Register	Value	Meaning
CX	**3702H**	
EDX	**Flags:** **00H - No flags** **02H - QF_KEEP** **04H - QF_HIDE**	 **Disallow deletion** **For system use only**

Returned Values

Register	Value	Meaning
EBX	**Mutex handle**	
ECX	**Error code:** **00H - No error** **07H - E_NoQHandle**	 **No unused queue handles**

Explanation

Call X_MXCreate to create a mutex. The system gives it a unique name (which has little use other than debugging) and a handle. During the lifetime of the mutex the handle is unique.

You can set the QF_KEEP amd QF_HIDE flag bits in DX. If set, QF_KEEP stops subsequent X_MXDelete calls from deleting the mutex. QF_HIDE makes the mutex hidden from applications. Only system processes can use it.

Any process in any domain on the same computer can use the handle to enter the mutex zone. You can easily communicate the handle to processes in the same domain. The easiest way to give it to processes in other domains is to create a queue with a known name and to send the mutex handle in a message to the queue.

Mutexes are a special kind of RTM queue, and you can therefore use the RTM queue inquiry functions to get the name and the handle of the first process waiting to enter a mutex that is in use.

X_MXCreate returns an error code if the system cannot allocate any more mutexes.

See Also: X_MXEnter, X_MXDelete, Z_QReaderGet, Z_QNameGet

X_MXDelete (3802H)

Function

Delete a mutex.

Entry Parameters

Register	Value
CX	**3802H**
EDX	**Mutex handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue** **0AH - E_QInUse**	 **No queue of this handle exists** **Queue is in use**

Explanation

X_MXDelete corresponds to X_MXCreate. You should call it when you have no further need for the mutex. The system data structures that the mutex used are freed up.

Do not call X_MXDelete until you are sure no process might still try to enter the mutex. It would be possible for another mutex to be allocated the same handle for an entirely different purpose.

If the mutex has the QF_KEEP flag set it cannot be deleted by X_MXDelete.

X_MXDelete returns an error code if the handle is invalid or the mutex is in use.

See Also: X_MXCreate

X_MXEnter (3902H)

Function

Enter a mutex zone.

Entry Parameters

Register	Value
CX	**3902H**
EDX	**Mutex handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **No queue of this handle exists**

Explanation

Call X_MXEnter when you need exclusive use of a resource protected by a mutex.

If some other process is in the mutex zone, you will be suspended until that process exits from it. Tasks suspended on a mutex are held in a FIFO list. If you do not want to be suspended, you may call X_MXEnterC instead of X_MXEnter. It immediately returns if the mutex is busy.

If your own process has already entered the same mutex, the effect of reentering is merely to increment a counter. Only when you have exited the same number of times as you entered will the mutex become available to another process. This allows you to be fairly liberal in using X_MXEnter so long as you are careful to call X_MXExit the same number of times.

X_MXEnter does not inhibit the dispatcher. If you need to ensure that the dispatcher does not run while you are in some critical region, you should use X_CritEnter and X_CritExit respectively.

X_MXEnter returns an error code if the handle is invalid.

See Also: X_MXEnterC, X_MXExit, X_PDispatch

X_MXEnterC (3A02H)

Function

Conditionally enter a mutex zone.

Entry Parameters

Register	Value
CX	**3A02H**
EDX	**Mutex handle**

Returned Values

Register	Value	
ECX	**Error code:** **00H - No error** **0AH - E_QInUse**	 **Zone entered successfully** **Another process is in the zone**

Explanation

X_MXEnterC behaves exactly as X_MXEnter described above, except that if some other task is in the zone the error code E_QInUse is returned immediately. The calling task cannot be suspended by calling X_MXEnterC.

You must be sure to test the value in CX on exit, because you may or may not be in the mutex zone. If you are not, you must not call X_MXExit.

See Also: X_MXEnter, X_MXExit

X_MXExit (3B02H)

Function

Exit from a mutex zone.

Entry Parameters

Register	Value
CX	**3B02H**
EDX	**Mutex handle**

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue** **0AH - E_QInUse**	 **Handle is invalid** **Another process is in the zone**

Returned Values

Explanation

You must call X_MXExit to release a mutex to other processes when you have completed the operation it is protecting.

Code a call to X_MXExit after every X_MXEnter and every successful X_MXEnterC.

X_MXExit returns the error code E_NoQueue if the handle is invalid, or E_QInUse if this process does not currently own the zone.

See Also: X_MXEnter, X_MXEnterC

Z_MemAlloc (0C03H)

Function

Allocate memory.

Entry Parameters

Register	Value
CX	**0C03H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Explanation"**

Returned Values

Register	Value	Meaning
EBX	**Limit of largest available block if E_NoMemory**	
ECX	**Error code:** **00H - No error** **03H - E_NoMemory**	 **No free memory**

Explanation

Z_MemAlloc allocates linear memory. Applications and device drivers should not normally use this low level interface. They should use XMS or DPMI instead. The parameter block is as described under Z_DescAlloc. You supply only DESC_PB_LIMIT and DESC_PB_MINFO. On return, if successful, the DESC_PB_BASE field will contain the base of the allocated block. Otherwise, EBX will contain the limit of the biggest available block.

In order to address the memory, you will need to call Z_DescAlloc or Z_DescSet.

Like Z_MemDescAlloc, you must supply valid flags in DESC_PB_MINFO. These are self-explanatory except, perhaps, for bit 5 which, if set, causes page tables to be created with the Present bit reset, so the linear memory space exists but no physical memory is mapped to it. If bit 5 is reset, physical memory is allocated and page tables set to point to it.

You can free up the memory by calling Z_MemFree.

See Also: Z_DescAlloc, Z_DescGet, Z_MemDescFree, Z_MemDescResize, Z_DescSet

Z_MemDescAlloc (1403H)

Function

Allocate memory and a descriptor pointing to it.

Entry Parameters

Register	Value
CX	**1403H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Explanation"**

Returned Values

Register	Value	Meaning
EBX	**Selector**	
ECX	**Error code:** **00H - No error** **03H - E_NoMemory** **31H - E_BadDesc** **32H - E_NoDesc**	 **No free memory** **Invalid DESC_PB_SINFO** **No free descriptors**

Explanation

Z_MemDescAlloc allocates memory and a protected mode descriptor pointing to it, so it combines the functions of Z_MemAlloc and Z_DescAlloc. The parameter block is as described under Z_DescAlloc. You supply all fields except DESC_PB_SEL, which will be filled in on a successful return.

Unlike Z_DescAlloc, you must supply valid flags in DESC_PB_MINFO. These are self-explanatory except, perhaps, for bit 5 which, if set, causes page tables to be created with the Present bit reset, so the linear memory space exists but no physical memory is mapped to it. If bit 5 is reset, physical memory is allocated and page tables set to point to it.

You can free both the memory and the descriptor by calling Z_MemDescFree.

See Also: Z_DescAlloc, Z_DescGet, Z_MemDescFree, Z_MemDescResize, Z_DescSet

Z_MemDescFree (1503H)

Function

Release a descriptor and the memory it points to.

Entry Parameters

Register	Value
CX	**1503H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **31H - E_BadDesc**	 **Invalid selector**

Explanation

Z_MemDescFree releases a protected mode memory descriptor and the memory block it points to. Thus it combines the functions of Z_MemFree and Z_DescFree. The parameter block is as described under Z_DescAlloc. Only the DESC_PB_SEL field need be filled in when you call Z_MemDescFree.

Note that freeing a descriptor renders its selector invalid. The selector must not be loaded into a segment register when the descriptor is freed, or subsequently reloaded.

See Also: Z_DescAlloc, Z_DescGet, Z_MemDescAlloc, Z_DescSet

Z_MemDescResize (1703H)

Function

Change the size of the memory pointed to by a descriptor.

Entry Parameters

Register	Value
CX	**1703H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
EBX	**Selector if no error, else limit of largest available block**	
ECX	**Error code:** **00H - No error** **03H - E_NoMemory** **31H - E_BadDesc**	 **No free memory** **Invalid selector**

Explanation

Z_MemDescResize attempts to change the size of a memory block pointed to by a descriptor. It may relocate the block to achieve this. The parameter block is as described under Z_DescAlloc. Only the DESC_PB_SEL and DESC_PB_LIMIT fields need be filled in when you call Z_MemDescResize.

On return, if successful, the parameter block's DESC_PB_BASE field will contain the base of the (possibly relocated) memory. The selector is unchanged. If the call is unsuccessful, the limit of the largest available block is returned in EBX.

See Also: Z_DescAlloc, Z_DescGet, Z_MemDescAlloc, Z_DescSet

Z_MemDescSizeGet (1603H)

Function

Get the size of a memory block pointed to by a descriptor.

Entry Parameters

Register	Value
CX	**1603H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **31H - E_BadDesc**	 **Invalid selector**

Explanation

Z_MemDescSizeGet returns the size of a memory block and its flags. The block must be one allocated by Z_MemAlloc or Z_MemDescAlloc. The parameter block is as described under Z_DescAlloc. Only the DESC_PB_SEL field needs to be filled in when you call Z_MemDescSizeGet.

On return, if successful, the parameter block's DESC_PB_BASE, DESC_PB_LIMIT and DESC_PB_MINFO fields will contain the base, limit and flags of the memory block.

See Also: Z_MemAlloc, Z_MemDescAlloc, Z_MemDescResize

Z_MemFree (0D03H)

Function

Release a block of memory.

Entry Parameters

Register	Value
CX	**0D03H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **1BH - E_BadMHandle**	 **Invalid base address**

Explanation

Z_MemFree releases a block of memory allocated by Z_MemAlloc or Z_MemDescAlloc. The parameter block is as described under Z_DescAlloc. Only the DESC_PB_BASE field need be filled in when you call Z_MemDescFree.

See Also: Z_DescAlloc, Z_DescGet, Z_MemDescAlloc, Z_DescSet

X_MemFreeGet (3103H)

Function

Get total free memory.

Entry Parameters

Register	Value
CX	**3103H**

Returned Values

Register	Value	
EBX	**Free memory**	
ECX	**Error code:** **00H - No error**	

Explanation

X_MemFreeGet returns the total amount of free memory in the system, in bytes. The task manager may be limiting the amount available to each domain, so call X_MemDomFreeGet if you need to know how much memory your application could allocate.

See Also: X_MemDomFreeGet

Z_MemResize (0F03H)

Function

Change the size of a memory block.

Entry Parameters

Register	Value
CX	**0F03H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
EBX	**Limit of largest available block if E_NoMemory**	
ECX	**Error code:** **00H - No error** **03H - E_NoMemory**	 **No free memory**

Explanation

Z_MemResize attempts to change the size of a memory block without relocating it. The memory should be a block which was allocated by Z_MemAlloc. The parameter block is as

described under Z_DescAlloc. Only the DESC_PB_BASE and DESC_PB_LIMIT fields need be filled in when you call Z_MemResize. The DESC_PB_LIMIT field is the requested size.

On return, if successful, the block's MINFO flags are written into DESC_PB_MINFO. If unsuccessful, the limit of the largest available block is returned in EBX.

See Also: Z_DescAlloc, Z_MemAlloc, Z_MemSizeGet

Z_MemSizeGet (0E03H)

Function

Get the size of a memory block.

Entry Parameters

Register	Value
CX	**0E03H**
DS:EDX	**Pointer to a Descriptor Parameter Block; see "Z_DescAlloc (1003H)" on page 2-18**

Returned Values

Register	Value	Meaning
EBX	**Selector if no error, else limit of largest available block**	
ECX	**Error code:** **00H - No error** **1BH - E_BadMHandle**	 **Invalid block base**

Explanation

Z_MemSizeGet returns the size of a memory block and its flags. The block must be one allocated by Z_MemAlloc or Z_MemDescAlloc. The parameter block is as described under Z_DescAlloc. Only the DESC_PB_BASE field needs to be filled in when you call Z_MemSizeGet.

On return, if successful, the parameter block's DESC_PB_LIMIT and DESC_PB_MINFO fields will contain the limit and flags of the memory block.

See Also: Z_MemAlloc, Z_MemDescAlloc, Z_MemResize

X_MemTopGet (3203H)

Function

Get top address in memory.

Entry Parameters

Register	Value
CX	**3203H**

Returned Values

Register	Value	
EBX	**Top address**	
ECX	**Error code:** **00H - No error**	

Explanation

X_MemTopGet returns the highest memory address in the system. This does not necessarily equal the amount of memory, because there may be holes in it, but no physical memory address can be higher than the number returned.

See Also: X_MemTotalGet

X_MemTotalGet (3303H)

Function

Get total extended memory size.

Entry Parameters

Register	Value
CX	**3303H**

Returned Values

Register	Value	
EBX	**Total memory**	
ECX	**Error code:** **00H - No error**	

Explanation

X_MemTotalGet returns the total number of bytes of extended memory in the system.

See Also: X_MemTopGet

Z_ModuleReg (0201H)

Function

Register an OS module.

Entry Parameters

Register	Value
EBX	**Module number**
CX	**0201H**
DS:EDX	**Pointer to a Module Descriptor; see "Explanation"**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **30H - E_BadModule**	 **Bad module number or 32-bit offset**

Explanation

Z_ModuleReg registers additional modules to those listed in Table 2-1 on page 2-2. To use it, you need information beyond the scope of this document.

The parameter is a pointer to a Module Descriptor that consists of a DESC_PBLK followed by the 32-bit offset of the entry point to the module. Only the DESC_PB_SEL and MODULE_OFFSET need be initialized.

DESC_PB_BASE	dd	?
DESC_PB_LIMIT	dd	?
DESC_PB_SEL	dw	?
DESC_PB_MINFO	db	?
DESC_PB_SINFO	db	?
MODULE_OFFSET	dd	?

Z_MoveReal (0501H)

Function

Relocate a segment into extended memory.

Entry Parameters

Register	Value
CX	**0501H**
DS:EDX	**Pointer to a Descriptor Parameter Block**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **03H - E_NoMemory** **31H - E_BadDesc** **32H - E_NoDesc**	 **No free memory** **Invalid DESC_PB_SINFO** **No free descriptors**
DS:[EDX]		**Parameter block filled in**

Explanation

Z_MoveReal allocates extended memory and copies the specified real mode data into it. You must initialize the parameter block with the linear base address and limit of the existing

real mode data, and flags to indicate the destination segment type and alignment. See page 2-18 for full details of Descriptor Parameter Blocks.

On return, memory and a descriptor will have been allocated, and the data moved from real to extended memory. In the DPB, DESC_PB_BASE will contain the base of the relocated segment, and DESC_PB_SEL its selector.

See Also: Z_MemDescAlloc

X_MsgDomEnter (1505H)

Function

Enter Domain Message Mode.

Entry Parameters

Register	Value
CX	**1505H**
DS:EDX	**Segment:Offset pointer to a CALL_PBLK**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **56FFH - E_MsgDomEnter**	 **Already in Domain Message Mode**

Explanation

X_MsgDomEnter allows programs such as TSRs to display messages without needing to save and restore the underlying screen contents or mode. You pass to X_MsgDomEnter a CALL_PBLK (as described under X_PCreate on page 2-126), which identifies a real-mode routine that will display your message and may accept keystrokes from the user.

If the caller's domain is currently in the background, X_MsgDomEnter immediately returns to the caller without calling the message display routine. When the domain is brought to the foreground, VM puts the screen into text mode and calls your message display routine. It can now use INT 10 or direct screen writes to display on the screen, but can use only page 0 and font 0, because that is all that has been saved. As the domain is in the foreground, it has the use of the keyboard and mouse.

The task manager is not disabled, so the user could switch you back into the background. If this happens, VM will call your message display routine each time your domain is again brought to the foreground, because it does not save the message screen. Eventually, you must call X_MsgDomExit, which ends Domain Message Mode.

E_MsgDomEnter is returned if the system is already in Domain Message Mode.

See Also: X_MsgGlobalEnter, X_MsgDomDisplay, X_MsgDomExit

X_MsgDomDisplay (1605H)

Function

Enter Domain Message Mode and display a message.

Entry Parameters

Register	Value
CX	**1605H**
DS:EDX	**Pointer to a message ASCIIZ string (maximum of 512 characters)**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **56FFH - E_MsgDomEnter**	 **Already in Domain Message Mode**

Explanation

X_MsgDomDisplay is similar to X_MsgDomEnter, but you pass it a pointer to a message, not a call-back routine.

If the caller's domain is currently in the background, X_MsgDomDisplay immediately returns to the caller without displaying the message. When the domain is brought to the foreground, VM puts the screen into text mode, blanks it and displays your message, which must not exceed 512 characters.

The message may contain bell (07H), BS (08H), LF (0AH) and CR (0DH) to position the cursor.

The task manager is not disabled, so the user could switch you back into the background. If this happens, VM will display your message each time your domain is again brought to the foreground. Eventually, your program must call X_MsgDomExit, which ends Domain Message Mode.

E_MsgDomEnter is returned if the system is already in Domain Message Mode.

See Also: X_MsgGlobalEnter, X_MsgDomEnter, X_MsgDomExit

X_MsgDomExit (1705H)

Function

Exit Domain Message Mode.

Entry Parameters

Register	Value
CX	**1705H**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **57FFH - E_MsgDomExit**	 **System not in Domain Message Mode**

Explanation

X_MsgDomExit reverses the effects of X_MsgDomEnter or X_MsgDomDisplay. It restores the previous screen contents and mode.

E_MsgDomExit is returned if the system is not in Domain Message Mode.

See Also: X_MsgDomDisplay, X_MsgDomEnter

X_MsgFatalDisplay (1805H)

Function

Display a fatal error message.

Entry Parameters

Register	Value
CX	**1805H**
DS:EDX	**Segment:Offset pointer to your message, which is an ASCIIZ string**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **41FFH - E_NoContext** **42FFH - E_DomNotExist** **43FFH - E_DomNotCreated** **44FFH - E_DomDeleted**	**See Appendix A, "Error Codes"**

Explanation

X_MsgFatalDisplay is identical to X_MsgGlobalDisplay, except that it does not save the current screen before it displays your message. The assumption behind this is that a fatal error has occurred from which the only escape will be to reboot the computer.

The various possible error codes should never be seen by applications.

See Also: X_MsgGlobalDisplay, X_MsgGlobalEnter, X_MsgGlobalExit, X_MsgDomDisplay

X_MsgGlobalDisplay (1305H)

Function

Enter Global Message Mode and display a message.

Entry Parameters

Register	Value
CX	**1305H**
DS:EDX	**Segment:Offset pointer to your message, which is an ASCIIZ string**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **41FFH - E_NoContext** **54FFH - E_MsgGlobalEnter**	**See "Explanation"**

Explanation

X_MsgGlobalDisplay is identical to X_MsgGlobalEnter, except that it displays your message before it returns to you.You can use INT 10 or direct screen writes to display further messages on the screen, but can use only page 0 and font 0, because that is all that has been saved. The task manager is disabled until you call X_MsgGlobalExit, so your message is guaranteed to be visible on the screen.

Your message may contain bell (07H), BS (08H), LF (0AH) and CR (0DH) to position the cursor.

E_MsgGlobalEnter is returned if the system is already in Global Message Mode.

E_NoContext should never be seen by applications.

You must call X_MsgGlobalExit to return to normal operation when the user has responded to your message.

See Also: X_MsgGlobalEnter, X_MsgGlobalExit, X_MsgDomDisplay

X_MsgGlobalEnter (1205H)

Function

Enter Global Message Mode.

Entry Parameters

Register	Value
CX	**1205H**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **41FFH - E_NoContext** **42FFH - E_DomNotExist** **43FFH - E_DomNotCreated** **44FFH - E_DomDeleted** **54FFH - E_MsgGlobalEnter**	**See Appendix A, "Error Codes"**

Explanation

X_MsgGlobalEnter brings the caller's domain to the foreground, saves the screen mode and contents, puts the screen into text mode and returns to the caller. The caller can now use INT 10 or direct screen writes to display on the screen, but can use only page 0 and font 0, because that is all that has been saved. The task manager is disabled until you call X_MsgGlobalExit, so your message is guaranteed to be visible on the screen.

E_MsgGlobalEnter is returned if the system is already in Global Message Mode.

The other possible error codes should never be seen by applications.

See Also: X_MsgGlobalDisplay, X_MsgGlobalExit, X_MsgDomEnter

X_MsgGlobalExit (1405H)

Function

Exit Global Message Mode.

Entry Parameters

Register	Value
CX	**1405H**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **42FFH - E_DomNotExist** **43FFH - E_DomNotCreated** **44FFH - E_DomDeleted** **55FFH - E_MsgGlobalExit**	**See Appendix A, "Error Codes"**

Explanation

X_MsgGlobalExit reverses the effects of X_MsgGlobalEnter or X_MsgGlobalDisplay. It restores the previous domain to the foreground and re-enables the task manager.

E_MsgGlobalExit is returned if the system is not in Global Message Mode.

The other possible error codes should never be seen by applications.

See Also: X_MsgGlobalDisplay, X_MsgGlobalEnter

Z_PCountGet (0102H)

Function

Get the number of times a process has run.

Entry Parameters

Register	Value
CX	**0102H**
EDX	**Process handle**

Returned Values

Register	Value	Meaning
EBX	**TickCount**	
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle**	 **Handle is invalid**

Explanation

Z_PCountGet returns the number of times the specified process has run. The act of calling Z_PCountGet resets the count to zero. The returned count is, strictly speaking, the number of times the dispatcher has been called when the process was running. The dispatcher is called every clock tick, and also when any process calls an RTM function that affects the list of ready processes (Z_PPriorSet, for example). However, for most purposes it is a reasonably accurate count of the number of clock ticks the process has run for.

See Also: X_PCreate, X_PTerm, X_PHandleGet, X_PUnsuspend, Z_TicksSet, X_TickGet

X_PCreate (0E02H)

Function

Create a process in the same domain.

Entry Parameters

Register	Value
CX	**0E02H**
DS:EDX	**Pointer to the PCREATE_PBLK of the new process**

Returned Values

Register	Value	Meaning
EBX	**Handle**	**Handle of child process**
ECX	**Error code:** **00H - No error** **03H - E_NoMemory** **0CH - E_NoPHandle** **32H - E_NoDesc** **35H - E_NoPage**	 **No more memory available** **No more Process Descriptors** **No more Descriptors** **No more memory pages**

Explanation

X_PCreate creates a new process or thread in the same domain as the calling process. Before returning, it runs the dispatcher. We call the created process a child process and the calling process its parent.

The parameter to the call is a pointer to a PCREATE_PBLK structure that specifies the initial values of all the registers with which the child will run. It also contains a name, the initial priority, and the state of the child process' flags. All of these are explained later in this section.

The format of a PCREATE_PBLK is defined in the supplied assembly language include file STRUCS.DEF. You must initialize the structure before calling X_PCreate.

The definition of PCREATE_PBLK is

PCREATE_PBLK	struc	
PCREATE_PB_CALLBLK	CALL_PBLK	
PCREATE_PB_NAME	db	8 dup (?)
PCREATE_PB_PRIOR	db	?
PCREATE_PB_FLAG	dw	?
	db	?
PCREATE_PBLK	ends	

where CALL_PBLK is another structure defined as

CALL_PBLK	struc	
CALL_PB_EDI	dd	?
CALL_PB_ESI	dd	?
CALL_PB_EBP	dd	?
Reserved	dd	?
CALL_PB_EBX	dd	?
CALL_PB_EDX	dd	?
CALL_PB_ECX	dd	?
CALL_PB_EAX	dd	?
CALL_PB_EIP	dd	?
CALL_PB_CS	dw	?
Interrupt	dw	?
CALL_PB_EFLAGS	dd	?
CALL_PB_ESP	dd	?
CALL_PB_SS	dw	?,?
CALL_PB_ES	dw	?,?
CALL_PB_DS	dw	?,?

CALL_PB_FS	dw	?,?
CALL_PB_GS	dw	?,?
CALL_PBLK	ends	

The layout of CALL_PBLK allows the child process's registers to be restored by pointing SS:ESP at the CALL_PBLK and executing POPAD followed by IRETD (for virtual 8086 mode processes). This is indeed how the dispatcher starts up virtual 8086 mode processes.

Child processes may run in virtual 8086 mode or protected mode at level 3. The VM_Flag bit (20000H) in CALL_PB_EFLAGS determines which is used.

If the child is to run in virtual 8086 mode you must give CALL_PB_CS and the other segment registers values appropriate to virtual 8086 mode. If the child will not use FS and/or GS their values do not matter. Set the high order word of ESP to zero. The system does not verify any of the values you pass to it, so be careful to ensure that they are valid.

If the child is to run in protected mode, you must give CALL_PB_CS and the other selectors values appropriate to level 3. The parent can use the high level DPMI interface to create and initialize selectors. Null (0000H) is an acceptable value for unused data segment selectors. The child will be running in the same domain as the parent, and you can therefore initialize its selectors to be aliases of the virtual 8086 memory.

The other fields in CALL_PBLK mimic the stack frame created by an interrupt followed by PUSHAD, so the Intel 80386 Programmer's Reference Manual is probably your best source of details. Note in particular the CALL_PB_EFLAGS field, which contains important flags. Note also that the segment registers and selectors really do have 4 bytes each.

Although a child process may share its parent's data segments, it must have its own stack, so be sure to allocate or reserve memory for this. CALL_PB_ESP should point to the top of this stack.

After CALL_PBLK, PCREATE_PBLK has three other fields.

- PCREATE_PB_NAME is an eight character name. Although not used by the system, the name may be useful if you need to tell the user what processes are running.
- PCREATE_PB_PRIOR is the initial priority of the child process. Set the priority in the range 200 (high) to 254 (low). See also Z_PPriorSet.
- PCREATE_PB_FLAG is the process flags word. The bits are defined as u bit 0: PF_SYS identifies system processes. Reset this bit for applications.

  u bit 1: PF_KEEP identifies processes that must not be terminated.

  u bits 2 to 15: reserved. Set them to zero.

  On return from this call, check CX for an error code. The codes listed are the most likely, but the list should not be taken as exhaustive. If the child's priority is at least as high as the parent's, it will start executing before the return to the parent.

X_PCreate returns the child process' 32-bit handle. This is a number that you can use to refer to the child process in subsequent calls.

See Also: Z_DomFork, Z_PPriorSet

X_PDelay (0B02H)

Function

Wait for a given number of ticks.

Entry Parameters

Register	Value
CX	**0B02H**
EDX	**Number of ticks**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error**	**No error is possible**

Explanation

X_PDelay puts the calling process to sleep for the specified number of clock ticks. The actual delay may be up to one tick more, depending on when the clock next ticks. When the time is up the process becomes ready to run again. Whether it runs immediately depends on whether there are any higher priority tasks running.

You can determine the clock tick period (usually about 55 milliseconds / 18 ticks per second) by calling X_TickGet.

See Also: X_PDispatch, X_TickGet

X_PDispatch (0C02H)

Function

Call the dispatcher.

Entry Parameters

Register	Value
CX	**0C02H**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error**	**No error is possible**

Explanation

X_PDispatch calls the dispatcher, which has the effect of allowing other processes of the same priority to run. The calling process will run again only when each of the other ready processes of the same priority has run. Call X_PDispatch if your process can do no more useful work immediately, but does not want to be suspended. It might, for example, need to monitor the state of something that does not generate an interrupt.

X_PDispatch is not portable to other multitasking and task switching DOS environments, and you may therefore prefer to call the equivalent standard function INT 2FH with AX=1680H, which returns AL=00H to indicate success.

The converse of X_PDispatch is to tell the system that the dispatcher must not run while you perform some sequence of operations. This can be done by calling X_CritEnter.

See Also: X_CritEnter, X_PDelay

X_PHandleGet (1002H)

Function

Get my process handle.

Entry Parameters

Register	Value
CX	**1002H**

Returned Values

Register	Value	Meaning
EBX	**Handle**	**The handle of this process**
ECX	**Error code:** **00H - No error**	**No error is possible**

Explanation

X_PHandleGet returns the handle of the calling process. The handle uniquely identifies the process.

There are calls to get the name, priority, status and domain of a process, given its handle. Calls that require a process handle allow the value of 0 to imply the calling process.

See Also: Z_PPriorGet, X_PTerm, Z_DomHandleGet, Z_PNameGet, Z_PStatusGet, X_PSuspend, X_PUnsuspend

Z_PHandleListGet (2802H)

Function

Get a list of all process handles.

Entry Parameters

Register	Value
CX	**2802H**
DS:EDX	**Pointer to a Handle List (HL) parameter block with valid HL_MAX**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error**	**No error is possible**
DS:[EDX]	**HL_ACTUAL** **HL_BUF**	**Number of processes** **Process handles**

Explanation

Z_PHandleListGet returns a list of the current process handles in a buffer supplied by the caller. The parameter you pass is a pointer to the following block:

HL_MAX	dw	BUFCNT
HL_ACTUAL	dw	?
HL_BUF	dd	BUFCNT dup (?)

Before calling Z_PHandleListGet, you must set the HL_MAX field to the number of handles the buffer will hold, which can be any number you choose. Z_PHandleListGet fills the buffer and puts the actual count in the HL_ACTUAL field. If there are more than HL_MAX processes, it returns as many as fit in the buffer.

See Also: X_PCreate, X_PTerm, X_PHandleGet, X_PUnsuspend

Z_PNameGet (2902H)

Function

Obtain the name of a process.

Entry Parameters

Register	Value
CX	**2902H**
DS:EDX	**Pointer to a parameter block**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle**	 **Handle is invalid**
	PD_NAME	**Process name**

Explanation

Z_PNameGet returns the name of a process, given its handle. The format of the parameter block is

PD_HANDLE	dd	?
PD_NAME	db	9 dup (?)

Before you call Z_PNameGet, you must set the PD_HANDLE field to the handle of the process whose name you are seeking. A process can obtain its own handle by calling X_PHandleGet, and can get a list of all processes by calling Z_PHandleListGet.

On return, the PD_NAME field contains the name of the process (8 characters) and a null terminator.

See Also: X_PCreate, X_PHandleGet, Z_PHandleListGet, X_PTerm

Z_PPriorGet (2C02H)

Function

Get the priority of a process.

Entry Parameters

Register	Value	Meaning
CX	**2C02H**	
EDX	**0** **Process handle**	**The calling process** **Another process**

Returned Values

Register	Value	Meaning
EBX	**Priority**	
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle**	 **Non-zero DX**

Explanation

Z_PPriorGet returns the current priority of the calling process (if EDX=0) or any other process. All applications have priorities in the range 200 (high) to 254 (low). System processes have higher priority.

See Also: Z_PPriorSet

Z_PPriorSet (0F02H)

Function

Set process priority.

Entry Parameters

Register	Value	Meaning
EBX	**Priority**	
CX	**0F02H**	
EDX	**0** **Process handle**	**The calling process** **Another process**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error**	 **No error is possible**

Explanation

Z_PPriorSet sets the priority of the calling process. Applications should use priorities in the range 200 (high priority) to 254 (low). By default, their priorities are all 200. System processes and interrupt handlers have higher priority.

Although this call returns no error code, it is not protected against an application that sets its priority to a very high level. Interrupt latency and system throughput depend on your cooperation.

See Also: Z_PPriorGet

Z_PStatusGet (2A02H)

Function

Get the status of a process.

Entry Parameters

Register	Value
CX	**2A02H**
EDX	**Process handle**

Returned Values

Register	Value	Meaning
BL	**Status**	
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle**	 **Handle is invalid**

Explanation

Z_PStatusGet returns the status of a process, given its handle. The status tells you whether the process is ready to run or not, and if not why not. This is probably of more interest to the system itself than to any application.

The values of the status byte are equated to symbolic names in the supplied assembly language include file STRUCS.DEF. The possible values are

0	PS_RUN	in ready list
2	PS_DELAY	in delay list
3	PS_FLAGWWTO	flag wait with timeout
4	PS_TERM	terminating
5	PS_SLEEP	suspended
6	PS_DQ	waiting to read a queue
7	PS_NQ	waiting to write to a queue
8	PS_FLAGWAIT	waiting for a flag set

The state of the calling process must be PS_RUN, since it is running.

See Also: X_PCreate, X_PHandleGet, Z_PNameGet, Z_DomHandleGet

X_PSuspend (2302H)

Function

Suspend a process.

Entry Parameters

Register	Value	Meaning
CX	**2302H**	
EDX	**0** **Process handle**	**Suspend the calling process** **Suspend another process**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle** **37H - E_AlreadyFrozen**	**Handle in EDX is invalid**

Explanation

X_PSuspend suspends the specified process. That means that the process is flagged as not being ready to run. The process will remain suspended until someone calls X_PUnsuspend or Z_DomUnsuspend to release it, or X_PTerm to terminate it.

You pass X_PSuspend the process handle of the process to be suspended. The parent of the process was told the handle when it created the process. A process can inquire its own handle by calling X_PHandleGet, but it can suspend itself by passing a handle of 0.

You cannot suspend processes with the PF_SYS or PF_KEEP flags set, but attempting to do so does not return an error.

See Also: X_PCreate, X_PTerm, X_PHandleGet, X_PUnsuspend

X_PTerm (1102H)

Function

Terminate a specified process.

Entry Parameters

Register	Value
EBX	**00000000H**
CX	**1102H**
EDX	**Process handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle** **23H - E_PNotTerminated**	 **Handle in EDX is invalid** **PF_KEEP or PF_SYS set**

Explanation

X_PTerm should be used to terminate processes that were created by X_PCreate. X_PTerm terminates a specified process, not necessarily the calling process. If you terminate the last process in a domain, the domain will be deleted.

If the terminated process is in any mutex zones, it is made to exit from them, so that they become available to other processes.

You pass X_PTerm the process handle of the process to be terminated. The parent of the process was told the handle when it created the process. A process can inquire its own handle by calling X_PHandleGet.

The call will fail if the process you try to terminate has either its PF_KEEP or PF_SYS flags set. Applications do not normally have either of these set. Processes can temporarily protect themselves against being terminated by calling X_PTermOff. The process that calls X_PTerm sees no error, and the target process will be terminated when it calls X_PTermOK.

See Also: Z_DomTerm, X_PCreate, X_PTermOff, X_PTermOK

X_PTermOff (1702H)

Function

Disable termination of processes.

Entry Parameters

Register	Value	Meaning
CX	**1702H**	
EDX	**0** **Process handle**	**Disable termination of calling process** **Disable termination of another process**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle**	 **Handle in EDX is invalid**

Explanation

X_PTermOff should be used to protect processes from being terminated by some other process calling X_PTerm. The corresponding call to reenable terminating is X_PTermOK. If someone called X_PTerm while this process was protected, it will be terminated when it calls X_PTermOK.

See Also: X_PCreate, X_PTermOff, X_PTerm

X_PTermOK (1802H)

Function

Reenable termination of processes.

Entry Parameters

Register	Value	Meaning
CX	**1802H**	
EDX	**0** **Process handle**	**Reenable termination of calling process** **Reenable termination of another process**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle**	 **Handle in EDX is invalid**

Explanation

X_PTermOK should be used to reenable terminating after X_PTermOff.

If someone calls X_PTerm while this process is protected, it will be terminated when it calls X_PTermOK.

See Also: X_PCreate, X_PTermOff, X_PTerm

X_PUnsuspend (2402H)

Function

Release a specified process.

Entry Parameters

Register	Value
CX	**2402H**
EDX	**Process handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **0BH - E_BadPHandle** **37H - E_NotFrozen**	 **Handle in DX is invalid** **Process is not suspended**

Explanation

X_PUnsuspend releases the specified process. That means that the process is flagged as being ready to run again.

You pass X_PUnsuspend the process handle of the process to be released.

See Also: X_PSuspend

Z_PageAlloc (0303H)

Function

Allocate a specific page of physical memory.

Entry Parameters

Register	Value
CX	**0303H**
EDX	**Physical page number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **3BH - E_PageAllocate**	 **Page already allocated**

Explanation

Z_PageAlloc allocates a specified page of physical memory, which you may map to linear memory as required. When you no longer need the memory, you should free it by calling Z_PageFree.

See Also: Z_PageFree, Z_PageLock, Z_PagesAlloc

Z_PageDomLock (2903H)

Function

Lock a page of linear memory, in a specified domain, with its existing contents.

Entry Parameters

Register	Value
EBX	**Domain handle**
CX	**2903H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
EBX	**Page table entry**	
ECX	**Error code:** **00H - No error** **33H - E_BadLock** **36H - E_BadDomain**	 **Already locked or No such page** **Invalid domain handle**

Explanation

Z_PageDomLock locks a specified page of memory in a specified domain so that it cannot be swapped to disk. If the memory is currently swapped to disk, its previous contents are restored from disk.

The page table entry is returned in EBX. The page table entry contains the base address of the page in physical memory and the Dirty, Accessed, User, R/W and Present flag bits, as defined by Intel Corporation.

See Also: Z_PageDomLockAny, Z_PageDomLockNone, Z_PageLock

Z_PageDomLockAny (2B03H)

Function

Lock a page of linear memory, in a specified domain, with undefined contents.

Entry Parameters

Register	Value
EBX	**Domain handle**
CX	**2B03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
EBX	**Page table entry**	
ECX	**Error code:** **00H - No error** **33H - E_BadLock** **36H - E_BadDomain**	 **Already locked or No such page** **Invalid domain handle**

Explanation

Z_PageDomLockAny locks a specified page of memory in a specified domain so that it cannot be swapped to disk. If the memory is currently swapped to disk, its previous contents are not restored from disk, but a page of memory is allocated. Use this function if you want to lock a page but do not care about its current contents.

The page table entry is returned in EBX. The page table entry contains the base address of the page in physical memory and the Dirty, Accessed, User, R/W and Present flag bits, as defined by Intel Corporation.

See Also: Z_PageDomLock, Z_PageDomLockNone, Z_PageUnlock

Z_PageDomLockNone (2D03H)

Function

Lock a page of linear memory, in a specified domain, with no physical memory.

Entry Parameters

Register	Value
EBX	**Domain handle**
CX	**2D03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
EBX	**Page table entry**	
ECX	**Error code:** **00H - No error** **33H - E_BadLock** **36H - E_BadDomain**	 **Already locked or No such page** **Invalid domain handle**

Explanation

Z_PageDomLockNone locks a specified page of memory in a specified domain so that it cannot be swapped to disk. Any physical memory currently mapped to this page is freed. This function is most used just before mapping a linear page to a specific physical address.

See Also: Z_PageDomLock, Z_PageDomLockAny, Z_PageUnlock

Z_PageDomUnlock (2A03H)

Function

Unlock a page of linear memory in a specified domain.

Entry Parameters

Register	Value
EBX	**Domain handle**
CX	**2A03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **33H - E_BadLock** **36H - E_BadDomain**	 **Already unlocked or No such page** **Invalid domain handle**

Explanation

Z_PageDomUnlock unlocks a specified page of memory in the specified domain so that it can be swapped to disk. It functions exactly as Z_PageUnlock, except that you specify which domain it should affect.

See Also: Z_PageDomLock, Z_PageDomUnlockNone, Z_PageDomUnlockReuse

Z_PageDomUnlockNone (2E03H)

Function

Unlock a page of linear memory in a specified domain. Do not reuse physical memory.

Entry Parameters

Register	Value
EBX	**Domain handle**
CX	**2E03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **33H - E_BadLock** **36H - E_BadDomain**	 **Already unlocked or no such page** **Invalid domain handle**

Explanation

Z_PageDomUnlockNone unlocks a specified page of memory in a specified domain and sets the page table entry to Not Present. Any physical memory that was mapped in is not reused. This call can be used to avoid having the same physical memory at two linear addresses.

See Also: Z_PageDomLock, Z_PageDomUnlock, Z_PageDomUnlockReuse, Z_PageUnlockNone

Z_PageDomUnlockReuse (2C03H)

Function

Unlock a page of linear memory, in a specified domain, and reuse it without swapping to disk.

Entry Parameters

Register	Value
EBX	**Domain handle**
CX	**2C03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **33H - E_BadLock** **36H - E_BadDomain**	 **Already unlocked or no such page** **Invalid domain handle**

Explanation

Z_PageDomUnlockReuse unlocks a specified page of memory in a specified domain and frees the physical memory without swapping it to disk. Use this function when the memory contents are no longer required.

See Also: Z_PageDomLock, Z_PageDomUnlock, Z_PageDomUnlockNone

Z_PageFree (0B03H)

Function

Release a page of physical memory.

Entry Parameters

Register	Value
CX	**0B03H**
EDX	**Physical page number**

Returned Values

Register	Value	Meaning
EBX	**Starting page number**	
ECX	**Error code:** **00H - No error** **34H - E_BadPage** **3AH - E_PageFree**	 **Bad page number** **Page already free**

Explanation

Call Z_PageFree when you no longer need memory allocated by Z_PagesAlloc. You must call Z_PageFree once for each page allocated.

See Also: Z_PagesAlloc

Z_PageLock (1803H)

Function

Lock a page of linear memory with its existing contents.

Entry Parameters

Register	Value
CX	**1803H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
EBX	**Page table entry**	
ECX	**Error code:** **00H - No error** **33H - E_BadLock**	 **Already locked or no such page**

Explanation

Z_PageLock locks a specified page of memory in the caller's domain so that it cannot be swapped to disk. If the memory is currently swapped to disk, its previous contents are restored from disk.

The page table entry is returned in EBX. The page table entry contains the base address of the page in physical memory and the Dirty, Accessed, User, R/W and Present flag bits, as defined by Intel Corporation.

See Also: Z_PageLockAny, Z_PageLockNone, Z_PageUnlock

Z_PageLockAny (1A03H)

Function

Lock a page of linear memory with undefined contents.

Entry Parameters

Register	Value
CX	**1A03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
EBX	**Page table entry**	
ECX	**Error code:** **00H - No error** **33H - E_BadLock**	 **Already locked or no such page**

Explanation

Z_PageLockAny locks a specified page of memory in the caller's domain so that it cannot be swapped to disk. If the memory is currently swapped to disk, its previous contents are not restored from disk, but a page of memory is allocated. Use this function if you want to lock a page but do not care about its current contents.

The page table entry is returned in EBX. The page table entry contains the base address of the page in physical memory and the Dirty, Accessed, User, R/W and Present flag bits, as defined by Intel Corporation.

See Also: Z_PageLock, Z_PageLockNone, Z_PageUnlock

Z_PageLockNone (1C03H)

Function

Lock a page of linear memory in the caller's domain, with no physical memory.

Entry Parameters

Register	Value
CX	**1C03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
EBX	**Page table entry**	
ECX	**Error code:** **00H - No error** **33H - E_BadLock**	 **Already locked or no such page**

Explanation

Z_PageLockNone locks a specified page of memory in the caller's domain so that it cannot be swapped to disk. Any physical memory currently mapped to this page is freed. This function is most used just before mapping a linear page to a specific physical address.

See Also: Z_PageLock, Z_PageLockAny, Z_PageUnlock

Z_PageUnlock (1903H)

Function

Unlock a page of linear memory in the caller's domain.

Entry Parameters

Register	Value
CX	**1903H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **33H - E_BadLock**	 **Already unlocked or no such page**

Explanation

Z_PageUnlock unlocks a specified page of memory in the caller's domain so that it can be swapped to disk.

See Also: Z_PageLock, Z_PageDomUnlock, Z_PageUnlockNone, Z_PageUnlockReuse

Z_PageUnlockNone (1D03H)

Function

Unlock a page of linear memory in the caller's domain. Do not reuse physical memory.

Entry Parameters

Register	Value
CX	**1D03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **33H - E_BadLock**	 **Already unlocked or no such page**

Explanation

Z_PageUnlockNone unlocks a specified page of memory in the caller's domain and sets the page table entry to Not Present. Any physical memory that was mapped in is not reused. This call can be used to avoid having the same physical memory at two linear addresses.

See Also: Z_PageDomUnlock, Z_PageLock, Z_PageUnlock, Z_PageUnlockReuse

Z_PageUnlockReuse (1B03H)

Function

Unlock a page of linear memory in the caller's domain, and reuse the physical memory without swapping it to disk.

Entry Parameters

Register	Value
CX	**1B03H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **33H - E_BadLock**	 **Already unlocked or no such page**

Explanation

Z_PageUnlockReuse unlocks a specified page of memory in the caller's domain and frees the physical memory without swapping it to disk. Use this function when the memory contents are no longer required.

See Also: Z_PageDomUnlockReuse, Z_PageLock, Z_PageUnlock, Z_PageUnlockNone

Z_PagesAlloc (0A03H)

Function

Allocate consecutive pages of physical memory.

Entry Parameters

Register	Value
CX	**0A03H**
EDX	**Number of pages required (1 page=4 KB)**

Returned Values

Register	Value	Meaning
EBX	**Starting page number**	
ECX	**Error code:** **00H - No error E_NoPage**	 **Not enough free memory**

Explanation

Z_PagesAlloc is provided for programs, such as system extensions, that need to use the lowest level memory allocation function. Applications should not use this call, but instead use XMS.

Z_PagesAlloc gives you consecutive pages, which you can map to linear memory as required. When you no longer need the memory, you should free it by calling Z_PageFree.

See Also: Z_PageAlloc, Z_PageFree

Z_ParallelBaseSet (1A05H)

Function

Set the base address of a parallel port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 2)**
CX	**1A05H**
DX	**Port Base Address**

Returned Values

Register	Value
ECX	**Error code:** **00H - No error** **5CFFH - E_LPTSet**

Explanation

Z_ParallelBaseSet sets the base address of the specified port. VM gets the default base address for LPT ports known to the ROM BIOS from the BIOS. Use this function if they need changing, or if LPT1 to LPT3 exist but are not known to the BIOS.

Z_ParallelBaseSet returns an error if the port number was outside the range 0 to 2, or if another process is setting the same port's configuration at exactly the same time. Note that LPT1 corresponds to port 0, LPT2 to port 1, and so on.

One base address value has a special meaning: 0 disables this port.

See Also: Z_ParallelIRQSet, Z_ParallelTimeoutSet, Z_SerialBaseSet

Z_ParallelIRQSet (1C05H)

Function

Set the IRQ number of a parallel port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 2)**
CX	**1C05H**
DL	**IRQ number (0 to 15)**

Returned Values

Register	Value
ECX	**Error code:** **00H - No error** **5BFFH - E_LPTSet**

Explanation

Z_ParallelIRQSet sets the IRQ number associated with the specified port. VM gets default IRQ numbers for LPT ports known to the ROM BIOS from the BIOS. Use this function if they need changing, or if LPT2 to LPT4 exist but are not known to the BIOS.

Z_ParallelIRQSet returns an error if the port number was outside the range 0 to 2, or if another process is setting the same port's configuration at exactly the same time. Note that LPT1 corresponds to port 0, LPT2 to port 1, and so on.

See Also: Z_ParallelBaseSet, Z_ParallelTimeoutSet, Z_SerialIRQSet

Z_ParallelOwnerGet (2305H)

Function

Get the handle of the domain that is using a parallel port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 2)**
CX	**2305H**

Returned Values

Register	Value	
EBX	**Domain handle**	

Explanation

Z_ParallelOwnerGet returns the handle of the domain that is using the specified parallel port.

See Also: Z_ParallelBaseSet, Z_ParallelIRQSet, Z_ParallelTimeoutGet, Z_ParallelTimeoutSet

Z_ParallelTimeoutGet (2405H)

Function

Get the timeout of a parallel port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 2)**
CX	**2405H**

Returned Values

Register	Value
EBX	**Timeout in milliseconds**

Explanation

Z_ParallelTimeoutGet gets a parallel port's timeout, which VM uses to determine when the caller's domain is no longer using the port. VM monitors port usage as described on page 1-25. The default timeout for all parallel ports is 15 seconds. Two timeout values have special meaning:

- 0 disables virtualization of this port. Any domain may use it at any time.
- -1 is treated as an infinite timeout. The first user will have exclusive use of the port. See Also: Z_ParallelBaseSet, Z_ParallelIRQSet, Z_ParallelTimeoutSet, Z_SerialTimeoutGet

Z_ParallelTimeoutSet (0F05H)

Function

Set the timeout of a parallel port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 2)**
CX	**0F05H**
EDX	**Timeout in milliseconds**

Returned Values

Register	Value
ECX	**Error code:** **00H - No error** **5CFFH - E_LPTSet**

Explanation

Z_ParallelTimeoutSet sets a parallel port's timeout, which VM uses to determine when the caller's domain is no longer using the port. VM monitors port usage as described on page 1-25. You should choose a timeout value of at least twice the maximum time you expect to elapse between any two accesses of the port, otherwise VM might give it away in the middle of an operation. The default timeout for all parallel ports is 15 seconds.

Z_ParallelTimeoutSet returns an error if the port number was outside the range 0 to 2, or if another process is setting the same port's configuration at exactly the same time. Note that LPT1 corresponds to port 0, LPT2 to port 1, and so on.

Two timeout values have special meaning:

- 0 disables virtualization of this port. Any domain may use it at any time. If this value is to be used, it must be called before the task manager runs.
- -1 is treated as an infinite timeout. The first user will have exclusive use of the port. See Also: Z_ParallelBaseSet, Z_ParallelIRQSet, Z_SerialTimeoutSet, Z_ParallelTimeoutGet

Z_PtblDomGet (0F04H)

Function

Get the page table entry of another domain.

Entry Parameters

Register	Value
CX	**0F04H**
DS:EDX	**Pointer to a parameter block; see "Explanation"**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **34H - E_BadPage** **36H - E_BadDomain**	 **Invalid page number** **Invalid domain handle**

Explanation

Z_PtblDomGet reads the current page directory and page tables of a specified domain. Note that in a virtual memory system the value returned may be meaningless unless the domain's page table entry has been locked.

The parameter block is defined as:

BANKPTBL_PBLK	struc	
BANKP_PB_DOMAIN	dd	?
BANKP_PB_ENTRY	dd	?
BANKP_PB_VALUE	dd	?
BANKPTBL_PBLK	ends	

Before you call Z_PtblDomGet, you must fill in the domain handle (BANKP_PB_DOMAIN) and page number (BANKP_PB_ENTRY) fields.

On return, the page table entry will have been copied into BANKP_PB_VALUE. The page table entry contains the base address of the page in physical memory and the Dirty, Accessed, User, R/W and Present flag bits, as defined by Intel Corporation.

See Also: Z_PtblDomSet, Z_PtblGet

Z_PtblDomSet (1004H)

Function

Set the page table entry of a specified domain.

Entry Parameters

Register	Value
CX	**1004H**
DS:EDX	**Pointer to a parameter block; see "Z_PtblDomGet (0F04H)" on page 2-167**

Returned Values

Register	Value	Meaning
EBX	**Page table entry**	
ECX	**Error code:** **00H - No error** **34H - E_BadPage** **36H - E_BadDomain**	 **Invalid page number** **Invalid domain handle**

Explanation

Z_PtblDomSet writes a page table entry of a specified domain. Note that in a virtual memory system you should lock the domain's page table before calling Z_PtblDomSet.

The parameter block is as defined under Z_PtblDomGet. Before you call Z_PtblDomGet you must fill in all three fields.

On return, the previous contents of the page table entry will have been copied into BANKP_PB_VALUE. The page table entry contains the base address of the page in physical memory and the Dirty, Accessed, User, R/W and Present flag bits, as defined by Intel Corporation.

See Also: Z_PtblDomSet, Z_PtblGet, Z_PtblSet

Z_PtblGet (0803H)

Function

Get a page table entry for the current domain.

Entry Parameters

Register	Value
CX	**0803H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
EBX	**Page table entry**	
ECX	**Error code:** **00H - No error** **34H - E_BadPage**	 **Invalid page number**

Explanation

Z_PtblGet reads the current page directory and page tables, and returns the page table entry for the page number requested. The page table entry contains the base address of the page in physical memory and the Dirty, Accessed, User, R/W and Present flag bits, as defined by Intel Corporation.

See Also: Z_PtblSet

Z_PtblSet (0903H)

Function

Set a page table entry for the current domain.

Entry Parameters

Register	Value
EBX	**Value to be set**
CX	**0903H**
EDX	**Linear page number**

Returned Values

Register	Value	Meaning
EBX	**Old page table entry**	
ECX	**Error code:** **00H - No error** **34H - E_BadPage**	 **Invalid page number**

Explanation

Z_PtblSet writes a new value into a page table entry. This may change the physical addresses corresponding to the page's linear address. This function is intended for the use of operating system components only.

The page table entry contains the base address of the page in physical memory and the Dirty, Accessed, User, R/W and Present flag bits, as defined by Intel Corporation. All of these are set by this call. The previous values are returned in EBX.

See Also: Z_PageLock, Z_PtblGet

X_QClose (3F02H)

Function

Close a queue.

Entry Parameters

Register	Value
CX	**3F02H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue** **0DH - E_QProtected**	 **No queue of this name exists** **Queue for system use only**

X_QClose closes a queue handle returned by X_QCreate or X_QOpen. It should be called when the calling process will make no further use of the handle.

See Also: X_QCreate, X_QDelete, X_QOpen

X_QCreate (0402H)

Function

Create a queue.

Entry Parameters

Register	Value
CX	**0402H**
DS:EDX	**Pointer to a QD (Queue Descriptor Block); see "Explanation"**

Returned Values

Register	Value	Meaning
EBX	**Queue handle**	
ECX	**Error code:** **00H - No error** **07H - E_NoQHandle** **08H - E_NoQBuf** **0AH - E_QInUse**	 **No system memory available** **No system memory available or Q_NMSGS = 0 is not allowed** **A queue with this name exists**

Explanation

X_QCreate creates a queue. The behaviour and uses of queues are described on page 1-7. The parameter you supply to this call is a pointer to a queue descriptor block with valid Q_FLAGS, Q_NAME, Q_MSGLEN and Q_NMSGS fields. This is used to initialize the queue.

The only Q_FLAGS flag you can specify is QF_KEEP (02H). If you do not set QF_KEEP, the queue may be deleted by someone calling X_QDelete. If QF_KEEP is set, the queue will not be deleted until the system is rebooted. All other flag bits should be zero.

The Q_NAME you supply can be any 8 bytes. All 8 are used in matching the name used in subsequent X_QOpen calls.

Q_MSGLEN is the message length in bytes. It is fixed for any one queue. A process can obtain a queue's message length by calling X_QMsgLenGet (2E02H).

Q_NMSGS is the number of messages the buffer is to hold. Note that the product of Q_MSGLEN and Q_NMSGS (which is the total buffer size) must be in the range 0 through 65536 bytes. A process can enquire Q_NMSGS by calling X_QMsgMaxGet (3002H). It can enquire how many messages are currently in a queue by calling X_QMsgNumGet.

Valid parameters lie within the following limits:

Field	Minimum	Maximum
Q_NAME	Any 8 bytes	Any 8 bytes
Q_FLAGS	0	QF_KEEP
Q_MSGLEN	0	65535 (bytes)
Q_NMSGS	1	65535 (messages)

If the call is successful, it creates the queue and allocates the buffer in the system data area. Other processes may open the queue by reference to its name, and then read from and write to it. The act of creating the queue opens it for the process that created it; this process need not, therefore, call X_QOpen, but should call X_QClose when it no longer needs to use the queue.

The format of a QD is defined in the supplied assembly language include file STRUCS.DEF. You can, for example, allocate and initialize memory for a QD for a queue to be called Example1 holding up to 100 messages of 128 bytes, as follows:

MyQD	QD	<`Example1',MYQFLAGS,128,100>

The definition in STRUCS.DEF is

QD	struc	
Q_NAME	ds	8
Q_FLAGS	dd	?
Q_MSGLEN	dd	?
Q_NMSGS	dd	?

QD	ends	

On return, check that CX is 0. If not, the queue has not been created because there was no memory available, or the Q_NMSGS was set to 0, or the queue already exists. The name must be unique across the system.

See Also: X_QClose, X_QDelete, X_QOpen, X_QRead, X_QWrite, and so on, to include all calls that start X_Q

X_QDelete (0602H)

Function

Delete a queue.

Entry Parameters

Register	Value
CX	**0602H**
DS:EDX	**Pointer to an 8 byte ASCII name**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue** **0AH - E_QInUse** **0DH - E_QProtected**	 **No queue of this name exists** **Queue is in use** **Queue for system use only**

Explanation

X_QDelete deletes a queue, so is the converse of X_QCreate.

X_QDelete can fail if the name is invalid, the queue is in use or is a system queue. A queue is in use if its buffer is not empty or any processes are waiting to read from it. If other processes have opened the queue but are not currently waiting on it, the deletion will take place. Opening a queue does not protect it against being deleted.

See the description of X_QCreate for details of the Queue name.

See Also: X_QCreate, X_QOpen

Z_QFlagsGet (3402H)

Function

Obtain a queue's flags.

Entry Parameters

Register	Value
CX	**3402H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
EBX	**FLAGS**	
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **No queue of this handle exists**

Explanation

Z_QFlagsGet returns the Flags word of the specified queue. The bits in this word have the following significance. The names are equates from the supplied assembly language include file STRUCS.DEF.

Bit	Name	Meaning if set
0	QF_MX	mutex
1	QF_KEEP	do not allow X_QDelete
2	QF_HIDE	for system use only

Applications need not know the uses of these flags, except to note that

- QF_MX identifies mutex queues. They have no buffer, but may have processes waiting to enter the zone. The first such process can be found by calling Z_QReaderGet.
- QF_KEEP means the queue cannot be deleted.

- QF_HIDE means the queue is for system use only. Applications cannot read or write it. You must call X_QOpen or Z_QHandleListGet to obtain the queue's handle before you call Z_QFlagsGet.

  See Also: X_QOpen, Z_QHandleListGet, Z_QReaderGet, Z_QWriterGet

Z_QFlagsSet (4502H)

Function

Set a queue's flags.

Entry Parameters

Register	Value
EBX	**FLAGS**
CX	**4502H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **No queue of this handle exists**

Explanation

The caller can alter the flags of the specified queue. The bits have the following significance:

- QF_KEEP means that the queue cannot be deleted.
- QF_HIDE means that the queue is for system use only. Applications cannot read or write it. See Also: X_QOpen, Z_QFlagsGet

Z_QHandleListGet (2D02H)

Function

Get a list of the handles of all queues.

Entry Parameters

Register	Value
CX	**2D02H**
DS:EDX	**Pointer to a Handle List (HL) parameter block with a valid HL_MAX**

Returned Values

Register	Value	Meaning
ECX	**Error codes: 00H - No error**	** No error is possible**
DS:[EDX]	**HL_ACTUAL HL_BUF**	**Actual number of processes Process handles**

Explanation

Z_QHandleListGet returns a list of the current queue handles in a buffer supplied by the caller. The parameter you pass is a pointer to the following block:

HL_MAX	dw	BUFCNT
HL_ACTUAL	dw	?
HL_BUF	dd	BUFCNT dup (?)

Before you call Z_QHandleListGet, you must set the HL_MAX field to the number of handles the buffer will hold, which can be any number you choose. Z_QHandleListGet fills the buffer and puts the actual count in the HL_ACTUAL field. If there are more than HL_MAX queues, it returns as many as fit in the buffer. The order of entries in the buffer is the order in which the queues were created, with the youngest first.

Given the handle of a queue, you can call Z_QNameGet to get its name, then X_QOpen to open it for reading and writing. You can also inquire all its attributes with the various X_Q...Get calls listed below.

See Also: X_QCreate, X_QOpen, X_QDelete, Z_QNameGet, X_QMsgLenGet, X_QMsgMaxGet, X_QMsgNumGet, Z_QFlagsGet, Z_QWriterGet, Z_QReaderGet

X_QMsgLenGet (2F02H)

Function

Obtain the message length of a queue.

Entry Parameters

Register	Value
CX	**2F02H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
EBX	**Q_MSGLEN**	**Message length in bytes**
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **No queue of this handle exists**

Explanation

X_QMsgLenGet returns a queue's message length, which is fixed at the time the queue is created by X_QCreate.

You must call X_QOpen or Z_QHandleListGet to obtain the queue's handle before you call X_QMsgLenGet.

See Also: X_QOpen, X_QMsgMaxGet, X_QMsgNumGet

X_QMsgMaxGet (3002H)

Function

Obtain the message capacity of a queue.

Entry Parameters

Register	Value
CX	**3002H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
EBX	**Q_NMSGS**	**Number of messages the buffer can hold**
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	**No queue of this handle exists**

Explanation

X_QMsgMaxGet returns the capacity of a queue buffer (not its length), which is fixed at the time the queue is created by X_QCreate.

You must call X_QOpen or Z_QHandleListGet to obtain the queue's handle before you call X_QMsgMaxGet.

See Also: X_QOpen, X_QMsgLenGet, X_QMsgNumGet

X_QMsgNumGet (3302H)

Function

Obtain the number of messages currently in a queue.

Entry Parameters

Register	Value
CX	**3302H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
EBX	**Q_MSGCNT**	**Number of messages in the queue**
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	**No queue of this handle exists**

Explanation

X_QMsgNumGet returns the actual number of messages in a queue.

You must call X_QOpen to obtain the queue's handle before you call X_QMsgNumGet.

See Also: X_QOpen, X_QMsgMaxGet, X_QMsgLenGet

Z_QNameGet (2E02H)

Function

Get the name of a queue or mutex.

Entry Parameters

Register	Value
CX	**2E02H**
DS:EDX	**Pointer to a parameter block**

Returned Values

Register	Value	Meaning
CX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **Handle is invalid**
DS:[EDX]	**QD_NAME**	**Mutex or Queue name**

Explanation

Z_QNameGet returns the name of a queue or mutex, given its handle. The format of the parameter block is

QD_HANDLE	dd	?
QD_NAME	db	9 dup (?)

Before you call Z_QNameGet, you must set the QD_HANDLE field to the handle of the queue whose name you are seeking. X_MXCreate and X_QCreate return the queue's handle, and you can get a list of all queues by calling Z_QHandleListGet.

On return, the QD_NAME field will contain the queue's name (8 characters) and a null terminator.

See Also: X_MXCreate, X_QCreate, Z_QHandleListGet

X_QOpen (0502H)

Function

Open a queue for reading, writing and/or deleting.

Entry Parameters

Register	Value
CX	**0502H**
DS:EDX	**Pointer to an 8 byte ASCII name**

Returned Values

Register	Value	Meaning
EBX	**Queue handle**	
ECX	**Error code:** **00H - No error** **09H - E_NoQueue** **0DH - E_QProtected**	 **No queue of this name exists** **Queue for system use only**

Explanation

X_QOpen opens a queue, which allows you to read or write its buffer, to obtain the message length and number, or to delete the queue. The parameter is a pointer to the name of the queue you wish to open. All eight bytes of the name must match those used in the X_QCreate call.

On return, if successful, EBX will contain the handle of the queue. The handle can then be used in calls to read, write or close the queue.

This call can fail if called from an application with the name of a system queue, or if no queue of that name exists.

See Also: X_QClose, X_QCreate, X_QDelete, X_QRead, X_QWrite

X_QRead (0702H)

Function

Read from a queue.

Entry Parameters

Register	Value
DS:EBX	**Pointer to a buffer**
CX	**0702H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **No queue of this handle exists**

Explanation

X_QRead reads a single message from a queue into your buffer. If there are no messages in the queue, your process is suspended until some other process writes to the queue. If several processes try to read from the same empty queue they are all suspended. The first such process will get the first message written to the queue.

You must call X_QOpen to obtain the queue's handle before you call X_QRead.

If the given handle is valid, this call will be successful, though you might have to wait a long time for a message. The related calls X_QReadC and X_QReadNDC allow you to get a message, if there is one, or to continue without if not. X_QRead and X_QReadC remove the message from the queue when they read it, whereas X_QReadNDC does not.

If you do not know the message length for this queue, you can enquire about it by calling X_QMsgLenGet (2F02).

See Also: X_QReadC, X_QMsgLenGet, X_QMsgNumGet, X_QOpen, X_QReadNDC, X_QWrite

X_QReadC (0802H)

Function

Conditionally read from a queue.

Entry Parameters

Register	Value
DS:EBX	**Pointer to a buffer**
CX	**0802H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue** **0EH - E_QEmpty**	 **No queue of this handle exists** **Queue is empty**

Explanation

If there are any messages in the queue, X_QReadC reads the first into your buffer. If there are no messages in the queue, the function returns immediately with the error code E_QEmpty. You can enquire about the number of messages in a queue by calling X_QMsgNumGet, but the number may change between when you do so and when you read the queue.

The related call X_QReadNDC is similar, but whereas X_QReadC removes the message from the queue, X_QReadNDC does not.

You must call X_QOpen to obtain the queue's handle before you call X_QReadC.

See Also: X_QRead, X_QReadNDC, X_QWrite, X_QWriteC

X_QReadNDC (1E02H)

Function

Non-destructively and conditionally read from a queue.

Entry Parameters

Register	Value
DS:EBX	**Pointer to a buffer**
CX	**1E02H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue** **0EH - E_QEmpty**	 **No queue of this handle exists** **Queue is empty**

Explanation

If there are any messages in the queue, X_QReadNDC copies the first into your buffer. If there are no messages in the queue, the function returns immediately with the error code E_QEmpty.

The related call X_QReadC is similar, but whereas X_QReadC removes the message from the queue, X_QReadNDC does not.

You must call X_QOpen to obtain the queue's handle before you call X_QReadNDC.

See Also: X_QOpen, X_QRead, X_QReadC, X_QWrite, X_QWriteC

Z_QReaderGet (3202H)

Function

Find out which process is waiting to read from a queue or enter a mutex.

Entry Parameters

Register	Value
CX	**3202H**
EDX	**Mutex or Queue handle**

Returned Values

Register	Value	Meaning
EBX	**00H** **Non-zero**	**No processes waiting** **Handle of first process**
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **No queue of this handle exists**

Explanation

Z_QReaderGet returns the handle of the first process waiting to read from the specified queue, or to enter the specified mutex. If there are none, it returns 0. If more than one process is waiting, the first process is the process that has been waiting longest.

For queues, you must call X_QOpen or Z_QHandleListGet to obtain the queue's handle before calling Z_QReaderGet. Mutex handles are returned by X_MXCreate.

See Also: Z_QFlagsGet, Z_QHandleListGet, X_QOpen, Z_QWriterGet

X_QWrite (0902H)

Function

Write to a queue.

Entry Parameters

Register	Value
DS:EBX	**Pointer to a buffer**
CX	**0902H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **No queue of this handle exists**

Explanation

X_QWrite writes a single message into a queue from your buffer. If the queue is already full, your process is suspended until some other process reads from the queue. If several processes try to write to the same full queue they are all suspended. The first such process will be the first to write to the queue when at last someone reads from it, thereby freeing some space. At that time it becomes ready to run again.

You must call X_QOpen to obtain the queue's handle before you call X_QWrite.

If the given handle is valid, this call will be successful, though you might have to wait a long time if the buffer is full. The related call X_QWriteC allows you to write a message if the buffer is not full, or to continue without writing if it is full.

If you do not know the message length for this queue, you can obtain it by calling X_QMsgLenGet (2F02).

See Also: X_QRead, X_QOpen, X_QMsgLenGet, X_QMsgNumGet, X_QWriteC

X_QWriteC (0A02H)

Function

Conditionally write to a queue.

Entry Parameters

Register	Value
DS:EBX	**Pointer to a buffer**
CX	**0A02H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **09H - E_NoQueue** **0FH - E_QFull**	 **No queue of this handle exists** **Queue is full**

Explanation

X_QWriteC writes a single message into a queue from your buffer, unless the queue is already full, in which case the message is not written and the error code E_QFull is returned. You can find out the capacity and number of messages currently in a queue by calling X_QMsgMaxGet and X_QMsgNumGet, but the number may change between when you do so and when you write to the queue.

You must call X_QOpen to obtain the queue's handle before you call X_QWriteC.

If you do not know the message length for this queue, you can find out by calling X_QMsgLenGet.

See Also: X_QRead, X_QOpen, X_QMsgLenGet, X_QMsgMaxGet, X_QMsgNumGet

Z_QWriterGet (3102H)

Function

Find out which process is waiting to write to a queue.

Entry Parameters

Register	Value
CX	**3102H**
EDX	**Queue handle**

Returned Values

Register	Value	Meaning
EBX	**00H** **non-zero**	**No processes waiting** **Handle of the first process**
ECX	**Error code:** **00H - No error** **09H - E_NoQueue**	 **No queue of this handle exists**

Explanation

Z_QWriterGet returns the handle of the first process waiting to write to the specified queue. If there are none, it returns 0. If more than one process is waiting, the first process is the process that has been waiting longest.

You must call X_QOpen or Z_QHandleListGet to obtain the queue's handle before you call X_QWriterGet.

See Also: X_QOpen, Z_QHandleListGet, Z_QReaderGet, Z_QFlagsGet

Z_Reboot (0601H)

Function

Reboot the computer.

Entry Parameters

Register	Value
CX	**0601H**

Returned Values

Register	Value	
EBX	**Total memory**	
ECX	**Error code:** **00H - No error**	

Explanation

Z_Reboot reboots the computer. Use it with discretion.

See Also: Z_MsgFatalDisplay

Z_SerialBaseSet (1905H)

Function

Set a serial port's base address.

Entry Parameters

Register	Value
EBX	**Port number (0 to 3)**
CX	**1905H**
EDX	**Port Base Address**

Returned Values

Register	Value
ECX	**Error code:** **00H - No error** **5BFFH - E_COMSet**

Explanation

Z_SerialBaseSet sets the base address of the specified port. VM gets default base addresses for COM1 to COM4 from the ROM BIOS. Use this function if these addresses need changing, or if COM1 to COM4 exist but are not known to the BIOS.

Z_SerialBaseSet returns an error if the port number was outside the range 0 to 3, or if another process is setting the same port's configuration at exactly the same time.

One base value has a special meaning: 0 disables this port.

See Also: Z_ParallelBaseSet, Z_SerialIRQSet, Z_SerialTimeoutSet

Z_SerialIRQSet (1B05H)

Function

Set the IRQ number of a serial port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 3)**
CX	**1B05H**
DL	**IRQ number (0 to 15)**

Returned Values

Register	Value
ECX	**Error code:** **00H - No error** **5BFFH - E_COMSet**

Explanation

Z_SerialIRQSet sets the IRQ number associated with the specified port. VM gets default IRQ numbers for COM1 to COM4 from the ROM BIOS. Use this function if these numbers need changing, or if COM1 to COM4 exist but are not known to the ROM BIOS.

Z_SerialIRQSet returns an error if the port number was outside the range 0 to 3, or if another process is setting the same port's configuration at exactly the same time.

See Also: Z_ParallelIRQSet, Z_SerialBaseSet, Z_SerialTimeoutSet

Z_SerialOwnerGet (2105H)

Function

Get the handle of the domain that is using a serial port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 3)**
CX	**2105H**

Returned Values

Register	Value
EBX	**Domain handle**

Explanation

Z_SerialOwnerGet returns the handle of the domain that is using the specified serial port.

See Also: Z_ParallelTimeoutSet, Z_SerialBaseSet, Z_SerialIRQSet, Z_SerialTimeoutGet, Z_SerialTimeoutSet

Z_SerialTimeoutGet (2205H)

Function

Get the timeout of a serial port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 3)**
CX	**2205H**

Returned Values

Register	Value
EBX	**Timeout in milliseconds**

Explanation

Z_SerialTimeoutGet gets a serial port's timeout, which VM uses to determine when the caller's domain is no longer using the port. VM monitors port usage as described on page 1-25. The default timeout for all serial ports is 5 seconds. Two timeout values have special meaning.

- 0 disables virtualization of this port. Any domain may use it at any time.
- -1 is treated as an infinite timeout. The first user will have exclusive use of the port. See Also: Z_ParallelTimeoutGet, Z_SerialBaseSet, Z_SerialIRQSet, Z_SerialTimeoutSet

Z_SerialTimeoutSet (0E05H)

Function

Set the timeout of a serial port.

Entry Parameters

Register	Value
EBX	**Port number (0 to 3)**
CX	**0E05H**
EDX	**Timeout in milliseconds**

Returned Values

Register	Value
ECX	**Error code:** **00H - No error** **5BFFH - E_COMSet**

Explanation

Z_SerialTimeoutSet sets a serial port's timeout, which VM uses to determine when the caller's domain is no longer using the port. VM monitors port usage as described on page 1-25. You should choose a timeout value of at least twice the maximum time you expect to elapse between any two accesses of the port, otherwise VM might give it away in the middle of an operation. The default timeout for all serial ports is 5 seconds.

Z_SerialTimeoutSet returns an error if the port number was outside the range 0 to 3, or if another process is setting the same port's configuration at exactly the same time.

Two timeout values have special meaning.

- 0 disables virtualization of this port. Any domain may use it at any time. If this value is to be used, it must be called before the Task Manager runs.
- -1 is treated as an infinite timeout. The first user will have exclusive use of the port. See Also: Z_ParallelTimeoutSet, Z_SerialBaseSet, Z_SerialIRQSet, Z_SerialTimeoutGet

Z_TMHotKeyDisable (0A05H)

Function

Disable task manager hot key scanning.

Entry Parameters

Register	Value
CX	**0A05H**

Returned Values

Register	Value	
ECX	**Error code:** **00H - No error**	

Explanation

Z_TMHotKeyDisable asks VM to stop scanning keyboard input for the task manager hot keys. Task managers normally call this function when their pop-up menu is on-screen, and call Z_TMHotKeyEnable when it goes away.

See Also: Z_TMHotKeyEnable, Z_TMInit

Z_TMHotKeyEnable (0905H)

Function

Enable task manager hot key scanning.

Entry Parameters

Register	Value
CX	**0905H**

Returned Values

Register	Value	
ECX	**Error code:** **00H - No error**	

Explanation

Z_TMHotKeyEnable informs VM that the task manager is loaded and wants VM to scan for the hot keys specified in Z_TMInit. The Task Manager calls this function when it has completed an operation and its pop-up window has been removed from the screen.

See Also: Z_TMHotKeyDisable, Z_TMInit

Z_TMHotKeyGet (0805H)

Function

Get the task manager wake-up hot key from VM.

Entry Parameters

Register	Value
CX	**0805H**
DS:EDX	**Segment:Offset pointer to a GETKEY_PBLK parameter block; see below**

Returned Values

Register	Value
CX	**Error code:** **00H - No error**

Explanation

The Task Manager calls Z_TMHotKeyGet to ask VM which hot key combination has been pressed by the user. VM fills in the GETKEY_PBLK provided by the task manager and returns.

You need to call this function only if you are writing a replacement task manager.

The parameter block is defined as follows:

GETKEY_PBLK	struc	
GETKEY_KEYTYPE	dw	?
GETKEY_HOTKEY	dw	?
GETKEY_PBLK	ends	

On return, if GETKEY_KEYTYPE contains 1, the key was found in the hot key table, and GETKEY_HOTKEY will contain the index into the hot key table of the keys actually pressed. If GETKEY_KEYTYPE does not contain 1, the task manager flag has been set but not in response to a valid hot key combination.

See Also: Z_TMInit

Z_TMInit (0705H)

Function

Tell VM the task manager hot key list.

Entry Parameters

Register	Value
CX	**0705H**
DS:EDX	**Segment:Offset pointer to a TMDATA_STRUC parameter block; see "Explanation"**

Returned Values

Register	Value
ECX	**Error code:** **00H - No error**

Explanation

The task manager calls Z_TMInit to inform VM which hot keys it wants to be flagged. The task manager then waits on the specified system flag. Whenever VM detects that the user has pressed one of the hot key combinations it sets the flag, which allows the task manager to run. The task manager can call Z_TMHotKeyGet to find out which hot key it was.

You need to call this function only if you are writing a replacement task manager.

The parameter block is defined as follows:

TMDATA_STRUC	struc	
TM_KEY_FLAG	dw	?
TM_HOTKEY_TBL	dw	24 dup (?)
TMDATA_STRUC	ends	

where each TM_HOTKEY_TBL entry has the scan code in the high byte and the ROM BIOS keyboard control shift bits in the low byte.

See Also: Z_FlagAlloc, Z_FlagWait, Z_TMHotKeyGet, Z_TMLoad

Z_TMLoad (0105H)

Function

Tell VM the task manager is loading.

Entry Parameters

Register	Value
CX	**0105H**

Returned Values

Register	Value	Meaning
ECX	**Error code:** **00H - No error** **46FFH - E_TMLoaded**	 **Task manager already loaded**

Explanation

The task manager calls Z_TMLoad to inform VM that it has been loaded into the initial domain. This alerts VM to initialize various features that will be needed when further domains are created.

Z_TMLoad fails if a task manager is already loaded.

You need to call this function only if you are writing a replacement task manager.

See Also: Z_TMInit, Z_TMUnload

Z_TMUnload (0205H)

Function

Tell VM the task manager is unloading.

Entry Parameters

Register	Value
CX	**0205H**

Returned Values

Register	Value	
ECX	**Error code:** **00H - No error** **45FFH - E_TMBusy** **47FFH - E_TMNotLoaded**	 **Task manager busy** **Task manager not loaded**

Explanation

The task manager calls Z_TMUnload to inform VM that it is about to be unloaded. With the supplied task manager this occurs when there is only one task running and the user asks for the task manager to be deleted. This function alerts VM to change various features back to their single-domain settings.

Z_TMUnload fails if there are more than one tasks running, or if the task manager is not loaded.

You need to call this function only if you are writing a replacement task manager.

See Also: Z_TMInit, Z_TMLoad

X_TickGet (3D02H)

Function

Obtain the clock tick period.

Entry Parameters

Register	Value
CX	**3D02H**

Returned Values

Register	Value
EBX	**Tick length in microseconds**
ECX	**Error code:** **00H - No error possible**

Explanation

X_TickGet tells you the clock tick period.

See Also: X_ForeCheck, Z_TicksSet, Z_PCountGet

Z_TicksSet (3C02H)

Function

Set the number of ticks per time slice for foreground processes.

Entry Parameters

Register	Value
CX	**3C02H**
EDX	**Slices**

Returned Values

Register	Value
EBX	**Previous value**
ECX	**Error code:** **00H - No error possible**

Explanation

Call Z_TicksSet to change the relative time devoted to executing foreground and background domains. Processes running in background domains are given one tick each time they run, whereas processes in the foreground domain are given several ticks. By default they are given five ticks, but you can change this by calling Z_TicksSet. You can

calculate the actual time in seconds if you also call X_TickGet to find out about the tick period.

This round robin scheduling applies only to processes of equal priority, and only when more than one is ready to run. No process of lower priority will run as long as there is at least one higher priority process ready.

Calling with EDX=0 allows examination of the current value without changing it.

See Also: X_ForeCheck, X_TickGet

Z_VMBackSet (0305H)

Function

Tell VM to save the console if enabled, then put to background.

Entry Parameters

Register	Value
CX	**0305H**

Returned Values

Register	Value	Meaning
CX	**Error code:** **0000H - No error** **41FFH - E_NoContext** **42FFH - E_DomNotExist** **43FFH - E_DomNotCreated** **44FFH - E_DomDeleted** **47FFH - E_TMNotLoaded** **48FFH - E_InSwitch** **53FFH - E_InGlobalMsgMode** **5FFFH - E_NoBuffersMode**	**See Appendix A, "Error Codes"**

Explanation

Z_VMBackSet is called by the task manager when the user has asked it to switch tasks. You need to call this function only if you are writing a replacement task manager. Z_VMBackSet saves the virtual console (unless saving has been disabled by Z_VMSaveDisable) so that it can be restored when the task next comes to the foreground. After a successful Z_VMBackSet, the task manager should call Z_VMForeSet to bring another domain to the foreground.

Z_VMBackSet fails if the current domain is invalid, if no task manager is loaded, if a switch is already taking place, or if Global Message Mode is active.

See Also: Z_VMForeSet, Z_VMSaveDisable

Z_VMForeSet (0405H)

Function

Tell VM to bring a specified domain to the foreground.

Entry Parameters

Register	Value
CX	**0405H**
EDX	**Domain handle**

Returned Values

Register	Value
ECX	**Error codes as for "Z_VMBackSet (0305H)" on page 2-215**

Explanation

Z_VMForeSet is called by the task manager when the user has asked it to switch tasks. You need to call this function only if you are writing a replacement task manager.

Z_VMForeSet restores the virtual console of the specified domain (unless saving has been disabled by Z_VMSaveDisable) and connects the keyboard and mouse to it.

Z_VMForeSet must be called after Z_VMBackSet. Z_VMForeSet can fail for the same reasons as Z_VMBackSet.

See Also: Z_VMBackSet, Z_VMSaveDisable

Z_VMSaveDisable (0605H)

Function

Tell VM to disable saving the virtual machine.

Entry Parameters

Register	Value
CX	**0605H**
EDX	**Domain handle**

Returned Values

Register	Value
ECX	**Error code as for "Z_VMBackSet (0305H)" on page 2-215**

Explanation

Z_VMSaveDisable may be called by a task manager to disable the saving of consoles when they are switched to the background. You need to call this function only if you are writing a replacement task manager.

Z_VMSaveDisable affects only the domain specified in EDX.

Z_VMSaveDisable fails if the task manager is not loaded, if a switch is taking place, or if Global Message Mode is currently active. See Z_VMBackSet for a list of error codes.

See Also: Z_VMBackSet, Z_VMSaveEnable

Z_VMSaveEnable (0505H)

Function

Tell VM to enable saving the virtual machine.

Entry Parameters

Register	Value
CX	**0505H**
EDX	**Domain handle**

Returned Values

Register	Value
ECX	Error code as for "Z_VMBackSet (0305H)" on page 2-215

Explanation

Z_VMSaveEnable may be called by a task manager to reenable the saving of consoles when they are switched to the background. You need to call this function only if you are writing a replacement task manager.

Z_VMSaveEnable affects only the domain specified in EDX.

Z_VMSaveEnable fails if the task manager is not loaded, if a switch is taking place, or if Global Message Mode is currently active. See Z_VMBackSet for a list of error codes.

See Also: Z_VMBackSet, Z_VMSaveDisable

---

info@caldera.com
Copyright © 1994, 1997, Caldera, Inc. All rights reserved.

Appendix A - Error Codes

Table A-1 lists the error codes that the multitasker can return in numerical order. Codes above 4000 hex are generated by the VM module, others may be generated by any module.

The error codes that can be anticipated have been listed under each function call. Other codes should never be generated by calls from applications, but it is difficult to be certain what might happen in extreme circumstances, so for completeness they are all listed here.

The file ERR.DEF provides assembly language equates between all the error names and their values.

Table A-1
Multitasker Error Codes

Error name	Value (hex)	Meaning
E_NotImplemented	0001	Not implemented
E_BadEntry	0002	Illegal function number
E_NoMemory	0003	Cannot find memory
E_BadFlag	0004	Illegal flag number
E_FlagOverrun	0005	Flag overrun
E_FlagUnderrun	0006	Flag underrun
E_NoQHandle	0007	No unused queue handles
E_NoQBuf	0008	No free queue buffers
E_NoQueue	0009	Cannot find queue on queue list
E_QInUse	000A	Queue in use
E_BadPHandle	000B	Process handle is bad
E_NoPHandle	000C	No free process handles
E_QProtected	000D	No queue access allowed
E_QEmpty	000E	Empty queue
E_QFull	000F	Full queue
E_NoMHandle	0012	No unused memory handle
E_NoPName	0014	Cannot find process in process list
E_BadMHandle	001B	Illegal memory handle
E_PNotTerminated	0023	Could not terminate process
E_FlagIgnored	002A	Flag set ignored

E_NoFlag	002D	No more system flags
E_FlagBusy	002E	Flag not in idle state
E_FlagTimeout	002F	Flag wait has timed out
E_BadModule	0030	Illegal module number
E_BadDesc	0031	Descriptor is bad
E_NoDesc	0032	No more descriptors
E_BadLock	0033	Error locking/unlocking a page
E_BadPage	0034	Error getting/setting page
E_NoPage	0035	No free pages
E_BadDomain	0036	Domain is invalid
E_AlreadyFrozen	0037	Process is already frozen
E_NotFrozen	0038	Process is not frozen
E_NoSwaplist	0039	Fork failure, no swaplist registered
E_PageFree	003A	Cannot free an already free page
E_PageAllocate	003B	Cannot allocate an already allocated page
E_Switcher	003C	Cannot switch tasks
E_CritUnderrun	003D	Freeing critical section we are not in
E_CritOverrun	003E	Critical sections nested too deeply
E_NoContext	41FF	No domain in context, or current domain invalid
E_DomainNotExist	42FF	Domain id is not a valid descriptor
E_DomainNotCreated	43FF	Domain creation not yet completed
E_DomainDeleted	44FF	Domain is being deleted
E_TMBusy	45FF	Cannot unload task manager while busy
E_TMLoaded	46FF	Cannot load task manager, already loaded
E_TMNotLoaded	47FF	Cannot unload task manager, not yet loaded
E_InSwitch	48FF	Cannot save/restore, previous switch not finished
E_CnsOwned	49FF	Cannot get/restore, console already has an owner
E_BggrOpcode	4BFF	Cannot decode opcode, not supported
E_BggrAddr32	4CFF	32-bit addressing prefix not supported
E_TimeoutRemove	4FFF	Timeout not on timer queue
E_TimeoutQlock	50FF	Could not lock timer queue
E_InGlobalMsgMode	53FF	Cannot switch while in Global Message Mode
E_MsgGlobalEnter	54FF	Error setting Global Message Mode
E_MsgGlobalExit	55FF	Not in Global Message Mode

E_MsgDomEnter	56FF	Error setting Domain Message Mode; system in this mode already
E_MsgDomExit	57FF	Not in Domain Message Mode
E_TimeoutAlloc	58FF	Could not allocate a timeout structure
E_VideoType	59FF	Cannot deal with video type
E_NotHandled	5AFF	No VM module handled the function
E_COMSet	5BFF	Error in "Serial..Set" call
E_LPTSet	5CFF	Error in "Parallel..Set" call
E_NoDom	5DFF	Domain list overflow
E_NoBuffersMode	5FFF	Cannot free domain because it is in nobuffers mode

info@caldera.com
Copyright © 1994, 1997, Caldera, Inc. All rights reserved.